

TECNOLOGÍAS DE LA INFORMACIÓN

Minería de Datos

3 créditos

Profesor:



Ricardo Ordoñez Avila, Mg.

Titulaciones	Semestre
MINERÍA DE DATOS	Sexto

Tutorías: El profesor asignado se publicará en el entorno virtual de aprendizaje online.utm.edu.ec), y sus horarios de conferencias se indicarán en la sección Cronograma de Actividades

PERÍODO ABRIL 2026 – AGOSTO 2026

Tabla de contenido

Tema 1: La minería de datos predictiva	5
1.1 Definiciones	5
1.2 Algoritmos de clasificación.....	14
1.3 Algoritmos de regresión.....	21
Tema 2: Técnicas de aprendizaje supervisado: clasificación y regresión. ...	28
2.1 Clasificación con Naive Bayes.....	28
2.2 Clasificación con Árbol de Decisión.....	37
2.2 Predicción con Análisis de Regresión.....	43
Tema 3: Métricas de calidad en modelos predictivos.	63
3.1 Matriz de confusión.....	63



Organización de la lectura para el estudiante por semana del compendio

Semanas	Compendio
Semana 5	Páginas 3 – 20
Semana 6	Páginas 21 – 36
Semana 7	Páginas 37 – 53
Semana 8	Páginas 53 – 64

Resultado de aprendizaje de la Unidad II

Demostrar conocimiento y capacidad de aplicación de los sistemas de extracción y de gestión del conocimiento relacionados con la predicción y clasificación.

Unidad II

Métodos Predictivos y Árboles de Decisión

La meta de la minería de datos es producir nuevo conocimiento que el usuario pueda utilizar. Esto se realiza construyendo un modelo del mundo real basado en los datos de una variedad de fuentes que puede incluir transacciones corporativas, historias de cliente e información demográfica, datos de control de proceso, y bases de datos externas pertinentes tal como información de agencia que suministra información acerca del crédito de presuntos clientes o datos del tiempo (Calderón, 2006). El resultado del modelo construido es una descripción de patrones y relaciones en los datos que se puede usar confiadamente para predicción.

Para evitar confundir los aspectos diferentes de la minería de datos, es necesario imaginar una jerarquía de las elecciones y decisiones que necesita hacer antes empezar:

- Meta de negocio
- El tipo de la predicción
- Tipo ejemplar
- Algoritmo
- Producto

El nivel más alto, es la meta de negocio: ¿cuál es el propósito de la minería de datos? Por ejemplo, buscando los modelos en sus datos para ayudarle a retener los clientes buenos, podría construir un modelo para predecir la rentabilidad de un cliente y un segundo modelo para identificar los clientes que probablemente pueden alejarse del negocio.

El conocimiento de las necesidades y objetivos de la organización facilitará la creación de metas en los modelos. El próximo paso está decidiendo el tipo de la predicción que es más apropiado. La clasificación, consiste en predecir en que categoría se clasifica cada caso. La regresión consiste en predecir el valor numérico que tendrá la variable (si es una variable que varía en el tiempo, ello se llama la predicción de serie cronológica).

En el ejemplo anterior, se podría usar regresión para predecir la cantidad de rentabilidad, y clasificación para predecir que clientes poseen una tendencia para alejarse del negocio. Ahora puede escoger el tipo ejemplar: una red neuronal para ejecutar la regresión, tal vez, y árbol de decisión para la clasificación. Existen también modelos estadísticos tradicionales para escoger, como logística regresión, análisis discriminante, o los modelos lineales generales.

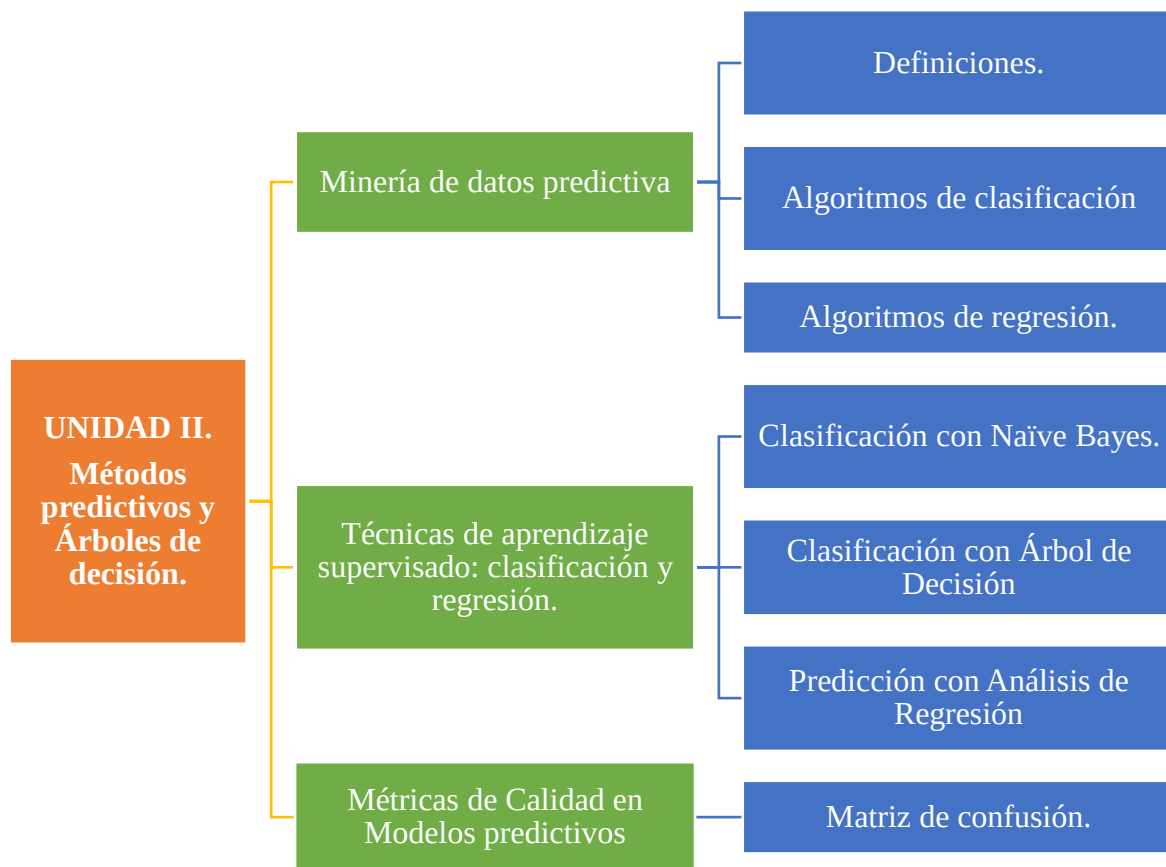
Muchos algoritmos son disponibles para construir sus modelos. Se puede construir la red neuronal usando backpropagation o funciones de base radiales. Para el árbol de decisión, se podría escoger entre CART, C5.0, QUEST, o CHAID.

Cuando se selecciona un producto de minería de datos, se debe tener presente que cada uno ha sido implementado con un algoritmo particular aun cuando lo identifican con el mismo nombre. En estas implementaciones las diferencias pueden afectar características operacionales tal como uso de memoria y almacenamiento de datos, así como características de ejecución tal como velocidad y exactitud.

Muchas metas de negocio deben ser construidas por modelos múltiples utilizando una variedad de algoritmos. Posiblemente no pueda ser capaz de determinar qué tipo modelo es mejor hasta que ha probado varios acercamientos (Calderón, 2006).

Finalmente, con base en su enfoque de aplicación y utilidad, en los últimos años el área predictiva ha tomado gran protagonismo en los negocios, la medicina, los servicios financieros, las políticas gubernamentales, la publicidad, la mercadotecnia, las redes sociales y gran cantidad de campos de aplicación.

Ejes Temáticos



Tema 1: La minería de datos predictiva

1.1 Definiciones

1.1.1 El análisis y los modelos predictivos

Un modelo predictivo es un conjunto de procesos ejercidos a través de técnicas computacionales de análisis de datos que ayudan a inferir la probabilidad de que ocurran determinadas situaciones previas a su consecución y, a su vez, detectar oportunidades de negocio. Los científicos de datos o Data Scientists centran sus esfuerzos en desarrollar esta rama del Big Data hasta el máximo nivel, ya que son los encargados de tratar los datos y convertirlos en información de valor para la empresa (Agencia B12, 2020).

Por su parte, Espino (2017), explica que un modelo predictivo es un mecanismo que predice el comportamiento de un individuo. Utiliza las características del individuo

como entrada y proporciona una calificación predictiva como salida. Cuanto más elevada es la calificación, más alta es la probabilidad de que el individuo exhiba el comportamiento predicho.

Los modelos predictivos son modelos de la relación entre el rendimiento específico de una unidad en una muestra y uno o más atributos o características conocidos de la unidad. El objeto del modelo es evaluar la probabilidad de que una unidad similar en una muestra diferente exhiba un comportamiento específico. Esta categoría abarca modelos que se encuentran en muchas áreas, como el marketing, donde buscan patrones de datos ocultos para responder preguntas sobre el desempeño del cliente, como los modelos de detección de fraude. Los modelos predictivos a menudo ejecutan cálculos durante las transacciones en curso, por ejemplo, para evaluar el riesgo o la oportunidad de un cliente o transacción en particular, de forma que aporte conocimiento a la hora de tomar una decisión.

Con los avances en la velocidad de computación, los sistemas de modelado de agentes individuales han sido capaces de simular el comportamiento humano o reacciones ante estímulos o escenarios específicos.

El análisis predictivo construye un modelo estadístico que utiliza los datos existentes para predecir datos de los cuales no se dispone. Como ejemplo del análisis predictivo se incluyen las líneas de tendencia o la puntuación de la influencia. Para la creación del modelo predictivo se utilizan unidades de muestra disponibles con atributos conocidos y un comportamiento conocido, a este conjunto de datos se le denomina conjunto de entrenamiento.

Por otro lado, se utilizará una serie de unidades de otra muestra con atributos similares, pero de las cuales no se conoce su comportamiento, a este conjunto de datos se le denomina conjunto de prueba. Con este antecedente, podemos plantearnos la pregunta: ¿Para qué sirve el análisis predictivo en una organización? En respuesta a esto, algunos enfoques son:

- *Reduce el impacto de riesgos:* Gracias al análisis predictivo, una organización puede identificar con antelación los riesgos que amenazan a la industria en la que se desarrolla. De esa manera, es más sencillo invertir

en acciones que ayudarán a evitar, o al menos minimizar, el daño para recuperarse en menos tiempo.

- *Optimiza las operaciones de una empresa:* Al predecir la cantidad de inventario que se necesitará, o identificar las necesidades que los clientes comenzarán a pedir del negocio, la empresa invierte los recursos de forma más inteligente. Gracias a la práctica del análisis predictivo es que las compras en línea se convirtieron en una de las ventajas de compañías como Amazon, que primero inició con la venta de libros y pronto descubrió que sería apenas la punta del iceberg, y así se convirtió en el éxito apabullante que es ahora.
- *Ayuda a crear estrategias de marketing más eficientes:* Además de adelantarse a una tendencia que está por convertirse en la norma, el análisis predictivo permite influir en las decisiones de los clientes, mostrando el contenido oportuno en el momento perfecto. Gracias a los datos disponibles sobre el comportamiento de un sector que todavía no toma una decisión sobre un producto o servicio, la empresa se da cuenta de la información que le hace falta para convencerle, a través de los canales en donde mejor los consumirá (Terreros, 2021).
- *Indica nuevos territorios para explorar:* Nos referimos a mercados o segmentos potenciales que quizá no eran viables en un inicio, pero que, gracias al crecimiento de la empresa y el rendimiento que tiene históricamente, son nuevas oportunidades para diversificarse y conquistar retos más importantes.
- *Muestra oportunidades de cross y up selling:* Para deleitar con mayor éxito a los clientes, una marca utiliza este tipo de análisis para descubrir quiénes agradecerán recibir noticias sobre ciertos artículos o servicios que complementan su experiencia, u ofrece algo totalmente innovador.

1.1.2 Tipos de modelos

Hay dos tipos de modelos predictivos: modelos de **clasificación** y de **regresión**. Los modelos de clasificación permiten predecir la pertenencia a una clase. Por ejemplo, si tratamos de clasificar entre nuestros clientes quiénes son más propensos al abandono. Los resultados del modelo son binarios, o un sí o un no (en

forma de 0 y 1) con su grado de probabilidad. Es decir, nos pueden decir que un cliente nos abandonará con el 89% de probabilidad. Los modelos de regresión en cambio nos permiten predecir un valor. Por ejemplo, cuál es el beneficio estimado que obtendremos de un determinado cliente (o segmento) en los próximos meses o nos ayudan a estimar el forecast de ventas (Keyrus, 2022). Marquez-Chamorro et al. (2018) como se citó en Zúñiga (2021), explica que las predicciones se pueden clasificar atendiendo a tres dimensiones:

- **Valor:** se puede clasificar dependiendo si el objeto de predicción es un valor categórico o numérico. Esta clasificación es útil porque los métodos empleados para construir el modelo y las métricas para evaluarlo generalmente dependen de estas categorías.
- **Alcance:** el valor predicho puede referirse solo a una parte del proceso, por ejemplo, el tiempo restante para terminar la etapa de ensamble en una línea de producción. También puede referirse a una agregación de varias etapas de un proceso; siguiendo con el ejemplo de la línea de producción, puede ser el tiempo total de manufactura de un solo tipo de producto.
- **Dominio (Aplicación):** dependiendo el fin último que se busca con la predicción, los resultados se pueden enfocar en cuatro diferentes dominios: indicadores de desempeño del proceso, predicciones de riesgo durante la ejecución, predicciones de violación de restricciones y otros valores predichos.

1.1.3 Etapas del proceso de análisis predictivo

- **Definición del proyecto:** Aquí debe tu organización determinar qué quiere alcanzar y cuáles serán las fuentes de datos que ayudarán a lograrlo.
- **Recolección de datos:** Es importante que consideren diferentes fuentes de información para nutrir el proceso con información valiosa y sustentada. Nos referimos a lo que se puede obtener gracias a la minería de datos, desde la información de medios y órganos oficiales de la industria hasta datos recopilados por sensores, sistemas de transacciones, registros en sitios web o centros de atención a clientes, entre otros.
- **Gestión de datos:** Será necesario contar con un sistema que ayude a «limpiar» la información, para deshacerse de lo que no es relevante para el proyecto en

particular y que solo contaminará las conclusiones. Por lo tanto, asegúrate de contar con un equipo que conozca cómo funciona la empresa, lo que busca y que tenga acceso a herramientas que hagan este proceso más llevadero, porque no es el más sencillo.

- **Análisis estadístico:** Es la primera parte del análisis de los datos, que arrojará información gracias a estadísticas descriptivas y que dará un estimado de ciertas probabilidades de comportamiento.
- **Modelación predictiva:** Aquí es donde creas y decides cuáles modelos predictivos entrarán en juego para tus objetivos.
- **Implementación del modelo:** Al poner en marcha el o los modelos predictivos, obtendrás resultados analíticos que podrás implementar en las áreas que lo necesiten. De esta forma tendrás resultados al automatizar las decisiones del día a día basándote en lo que el paso anterior puso en evidencia.
- **Supervisión del modelo:** Por supuesto, es necesario que des seguimiento al modelo o modelos para asegurar que no haya errores. Incluso podrías optimizarlo si así lo consideras; es buena idea no dejar que corra sin una buena vigilancia.

1.1.4 Herramientas para el análisis predictivo

Es importante considerar el uso potencial de una herramienta que haga el trabajo pesado con el análisis predictivo. Por fortuna existen varias, entre estas R y Python como lenguajes de programación.

Otras herramientas como, IBM son adecuadas para hacer análisis predictivo, esta en particular dispone de un set de herramientas que abarcan tres etapas clave. Primero, incluye la comprensión y análisis de datos con IBM SPSS Statistics, cuya función es que el usuario entienda la información, prediga y planifique hipótesis y llegue a conclusiones valiosas. Luego, IBM SPSS Modeler pone a la mano algoritmos y modelos de datos perfectos para los amantes de la información. Finalmente, Watson Studio Desktop simplifica el proceso de implementación y experimentación a partir de datos para que el negocio aproveche mejor la inteligencia artificial (Terreros, 2021).

Por su parte, Alteryx, es una herramienta que ayuda a recopilar información al conectarse con varias fuentes de datos y también hace una limpieza para preparar o

combinar lo que obtiene. Es un poco más amigable porque está dirigida administradores, no desarrolladores, además de que da la opción de personalizar informes y se preocupa por encontrar soluciones sencillas para sus clientes. En su mismo sitio web explica que podrás «automatizar procesos, insertar la toma de decisiones inteligentes y motivar a tus empleados para que brinden mejores y más veloces resultados empresariales».

DataBricks, es una opción ideal para las empresas que gestionan grandes volúmenes de datos. Esta, posee una variedad de herramientas con código abierto, además de otras como cuadernos de colaboración y embudos de procesamiento de datos que garantizan un trabajo fluido entre varios equipos.

DataRobot, Para quien gestiona sus datos y modelos predictivos en equipos locales, en la nube o en ambos, DataRobot ofrece soluciones que se centran en industrias como marketing, seguros, venta al menudeo y comunicaciones. Tiene herramientas para los distintos modelos de análisis predictivos desde un solo sitio para una mejor gestión.

MathWorks dispones de soluciones para el análisis de datos y estadísticas. MATLAB es la plataforma que pone al alcance de los expertos la gestión de información, con la opción de crear sus propios algoritmos y correr modelos. Gracias a sus apps, puedes comprobar cómo funcionan tus algoritmos y luego ver la información que resulta en gráficas, o compártelas directamente en la nube.

Finalmente, como referencia de algunas herramientas que son usadas para este tipo de tareas, está RapidMiner, la cual desea que sus usuarios tengan un trabajo más sencillo, por eso son transparentes en sus procesos. Sus herramientas de modelado están pensadas para que se automaticen y que quien los cree no necesite mucha ayuda. Además, ofrece un programa de certificación para aumentar las habilidades de las personas que no son científicas de datos. Su promesa es que los datos que obtendrás con esta plataforma son para expertos en información, aunque simplificados para todos los perfiles.

1.1.5 Aplicaciones de los modelos predictivos.

Si bien los modelos predictivos son aplicables en diferentes ámbitos, es en el ámbito de la estrategia digital donde se están generando grandes avances debido a la aparición de herramientas de Business Intelligence y el desarrollo de la inteligencia artificial. En marketing, por ejemplo, se utilizan para predecir el comportamiento de los usuarios y evaluar el margen de acierto que se puede alcanzar con determinadas acciones en el mercado.

Se trabaja cada vez más en modelos que permitan predecir, en base a lo que ha sucedido, qué sucederá en el futuro. Todo ello tiene aplicación directa en las *campañas de marketing*, con el objetivo de saber cómo van a funcionar incluso antes de haberlas lanzado (Arimetrics, 2022).

A continuación, se adhieren algunas de las áreas de aplicación de la analítica predictiva, diferenciando cinco áreas a través de las cuales se hace más rentable la cartera de clientes:

1. Segmentación de clientes. La segmentación permite adecuar las ofertas en función del nivel de ingresos, franja de edad, sexo o estudios realizados, entre otras variables.
2. Personalización de la oferta. Conocer cuál es la siguiente mejor oferta que se le puede hacer a un cliente a partir de su comportamiento histórico. “El cliente ha comprado un nuevo móvil, puede estar interesado en comprar una funda”.
3. Detectar el riesgo de que el cliente abandone la relación comercial en función del ritmo de pedidos o contactos que realiza o de las incidencias que registra.
4. Conocer cuáles son los clientes más propensos a responder a las iniciativas de comunicación publicitaria, para sacar el mayor provecho a la inversión hecha mercado lógicamente.
5. Conocer la tasa de deserción; es decir, predecir en forma anticipada y proactiva cuáles son los clientes que están buscando otras ofertas para evitar que estos desvíen su atención hacia la competencia. A través de esta aplicación separo los clientes rentables de los que no lo son.

Estos cinco casos están centrados en el sector empresarial y en el beneficio que el análisis predictivo le puede aportar al mismo, sin embargo, existen otros sectores que se pueden aprovechar de las ventajas del análisis predictivo. A continuación, se muestran algunos de los sectores que se benefician del análisis predictivo, así como aplicaciones que se utilizan en cada uno de ellos.

Sector de Marketing

Marketing directo: Consiste en un modelo capaz de predecir qué clientes responderán ante un contacto de marketing. Esto permite a las empresas comunicarse con aquellos clientes que tienen una mayor probabilidad de responder. Publicidad predictiva Consiste en un modelo capaz de predecir qué anuncio es más probable que cada cliente haga un clic. Esto permite a las empresas elegir el mejor anuncio basándose en la probabilidad de que el cliente haga clic y en lo que recibe por cada clic.

Predictor de embarazos: Consiste en un modelo capaz de predecir qué clientas van a tener un bebé en los próximos meses. Esta información permite a las empresas realizar ofertas relevantes para los futuros padres, como, por ejemplo, oferta de cunas o carritos de bebe.

Persuasión del voto en campañas electorales: Consiste en un modelo capaz de predecir qué votantes se pueden persuadir positivamente durante la campaña mediante contacto (llamada, anuncio de televisión, visita en la casa...). De este modo se pueden centrar los esfuerzos durante la campaña para acceder a aquellos votantes que pueden cambiar de voto (Espino, 2017).

Sector Actuarial

Detector de fraude: Consiste en un modelo capaz de predecir qué transacciones o solicitudes de crédito o reembolso tienen mayor probabilidad de ser fraudulentas, para que posteriormente sean analizadas con detenimiento.

Sector de servicios financieros

Compraventa de acciones. Consiste en un modelo capaz de predecir si una acción subirá o bajará. De este modo, al usuario de la aplicación obtendrá información

acerca de la probabilidad de que determinadas acciones suban, y así comprarlas, o bajen, y venderlas si es dueño de las mismas.

Estimación del valor hipotecario. Consiste en un modelo capaz de predecir qué clientes van a hacer el pre-pago de una hipoteca en un futuro cercano, de este modo pueden decidir si vender la hipoteca a otro banco o no.

Sector de Administraciones públicas

Reducción de reincidencia. Consiste en un modelo capaz de predecir la probabilidad de que un criminal al que se está enjuiciando pueda delinquir de nuevo. Los jueces y los tribunales pueden consultar las predicciones del modelo para tomar una decisión más correcta sobre el encarcelamiento de un individuo.

Sector Empresarial

Retención de clientes: Consiste en un modelo capaz de predecir qué clientes tienen mayor probabilidad de abandonar a la empresa. De este modo las empresas pueden orientar sus esfuerzos en retener a dichos clientes.

Recomendaciones de películas/canciones. Consiste en un modelo capaz de predecir qué puntuación le dará una persona a una película/canción. Gracias a esta aplicación, las empresas como Netflix o Spotify pueden realizar recomendaciones a clientes sobre películas/canciones que tengan una alta probabilidad de gustarle.



Sabías que: En los problemas de regresión los valores de los conjuntos que se desean reproducir han de tener un valor continuo. Siendo este el valor que se desea reproducir. Para ello se seleccionan las características con las que se realizarán las predicciones. A las que se denominan variables independientes. Una vez realizado esto se ha de definir una fórmula matemática y calcular los parámetros de modo que al introducir las características se obtenga el valor deseado. Esta se conoce como variable dependiente (Rodríguez, 2018).

1.2 Algoritmos de clasificación

A pesar de las diferencias metodológicas y matemáticas entre los tipos de modelos, el objetivo general de todos ellos es similar: predecir resultados futuros basándose en datos pasados. Aunque hay algunas técnicas que son específicas de clasificación y otras de regresión, la mayoría de las técnicas funcionan con ambos.

Árboles de clasificación o decisión

Son modelos de clasificación muy utilizados que tratan de encontrar la variable que permita dividir el dataset en grupos lógicos que son más diferentes entre sí. Cada árbol se va descomponiendo en distintas ramas y hojas que representan cada clasificación en función de las condiciones que se van seleccionando hasta llegar a la resolución del problema. Estos modelos son de gran ayuda a la hora de determinar las decisiones a lo largo de un proceso como por ejemplo el funnel de compra (Keyrus, 2022).

En los árboles de decisión se encuentran los siguientes componentes: nodos, ramas y hojas. Los nodos son las variables de entrada, las ramas representan los posibles valores de las variables de entrada y las hojas son los posibles valores de la variable de salida. Como primer elemento de un árbol de decisión tenemos el nodo raíz que va a representar la variable de mayor relevancia en el proceso de clasificación.

Todos los algoritmos de aprendizaje de los árboles de decisión obtienen modelos más o menos complejos y consistentes respecto a la evidencia, pero si los datos contienen incoherencias, el modelo se ajustará a estas incoherencias y perjudicará su comportamiento global en la predicción, es lo que se conoce como sobreajuste. Para solucionar este problema hay que limitar el crecimiento del árbol modificando los algoritmos de aprendizaje para conseguir modelos más generales. Es lo que se conoce como poda en los árboles de decisión (Parra, 2019).

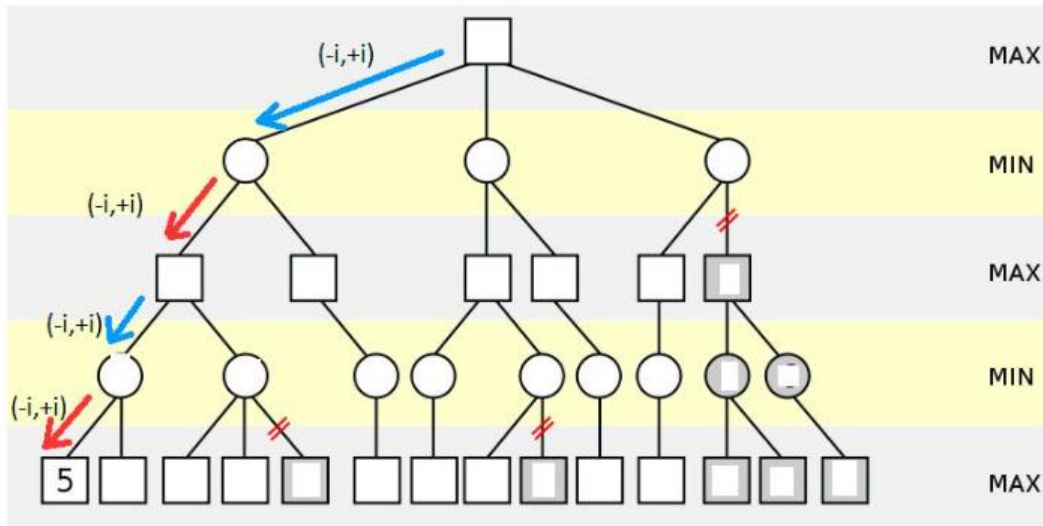


Figura 1. Árboles de decisión (Parra, 2019).

Las reglas de parada tratan de preguntar si merece la pena seguir o detener el proceso de crecimiento del árbol por la rama actual, se denominan reglas de pre poda ya que reducen el crecimiento y complejidad del árbol mientras se está construyendo:

- Purez de nodo. Si el nodo solo contiene ejemplos o registros de una única clase se decide que la construcción del árbol ya ha finalizado.
- Cota de profundidad. Previamente a la construcción se fija una cota que nos marque la profundidad del árbol, cuando se alcanza se detiene el proceso.
- Umbral de soporte. Se especifica un número de ejemplos mínimo para los nodos, y cuando se encuentre un nodo con ejemplos por debajo del mínimo se para el proceso, ya que no consideramos fiable una clasificación abalada con menos de ese número mínimo de ejemplos.

Existen dos formas de poda muy comunes utilizadas en los diferentes algoritmos: la poda por coste-complejidad y la poda pesimista. En la poda por coste-complejidad se trata de equilibrar la precisión y el tamaño del árbol. La complejidad está determinada por el número de hojas que posee el árbol (nodos terminales). La poda pesimista utiliza los casos clasificados incorrectamente y obtiene un error de sustitución, eliminando los subárboles que no mejoran significativamente la precisión del clasificador (Parra, 2019)..

Características de los algoritmos de clasificación

Los algoritmos que se encuentran, o bien solos o bien integrados en diferentes paquetes informáticos, son los que determinan o generan el procedimiento de cálculo que establece el orden de importancia de las variables en cada interacción. También se pueden imponer ciertas limitaciones en el número de ramas en que se divide cada nodo. Los elementos y las herramientas de los algoritmos que determinan la construcción de un árbol son varios:

- El criterio para determinar la partición de cada nodo.
- La regla que declara un nodo terminal.
- La asignación de una clase a cada nodo terminal, lo que determina la regla de clasificación.
- Fusión: En relación a la variable dependiente, las categorías de las variables predictoras no significativas se agrupan juntas para formar categorías combinadas que sean significativas.
- Partición. Selección del punto de división. La variable utilizada para dividir el conjunto de todos los datos se elige por comparación con todas las demás.
- Poda. Se eliminan las ramas que añaden poco valor de predicción al árbol.
- La evaluación de la bondad del clasificador obtenido. La estimación de la validación del árbol y el cálculo del riesgo. Los métodos utilizados son los mismos, independientemente del método que se utilice para la generación del árbol.

Árbol CHAID (CHi-square Automatic Interaction Detection) y CHAID exhaustivo. El algoritmo en el que se basa el CHAID, el AID (Automatic Interaction Detection) o Detección Automática de Interacciones, fue uno de los más utilizados en la década de los años setenta y principios de los ochenta hasta que surgió el CHAID. Se le llama así porque la idea inicial no perseguía el objetivo de la clasificación, sino que estaban centrados en las interacciones entre las variables (Parra, 2019)..

Árbol CART (Classification and Regression Trees). El algoritmo CART es el acrónimo de Classification And Regression Trees (Árboles de Clasificación y de Regresión) fue diseñado por Breiman et al. (1984). Con este algoritmo, se generan

árboles de decisión binarios, lo que quiere decir que cada nodo se divide en exactamente dos ramas.

Árbol QUEST (Quick, Unbiased, Efficient Statistical Tree). Este procedimiento denominado QUEST es el acrónimo de Quick, Unbiased, Efficient Statistical Tree (Árbol Estadístico Eficiente, Insesgado y Rápido). Este método fue propuesto por Loh y Shih (1997). Este algoritmo trata de corregir y de restringir la exhaustiva búsqueda de particiones significativas que se generan tanto en los algoritmos AID y CHAID como en el CART. Este método selecciona de forma previa la variable que segmenta mejor los datos, y después realiza la división óptima de ella.

Árbol C5.0. El algoritmo C5 y, sobre todo, su versión no comercial, C4.5, es uno de los algoritmos más utilizados en el ámbito de los árboles de clasificación. La forma de inferir árboles de decisión a través de este algoritmo es el resultado de la evolución del algoritmo C4.5 (Quinlan, 1993) diseñado por el mismo autor y que a su vez es el núcleo del programa perteneciente a la versión ID3 (Quinlan, 1986b). Este algoritmo crea modelos de árbol de clasificación, permitiendo sólo variables de salida categórica. Las variables de entrada pueden ser de naturaleza continua o categórica.

Máquina Soporte Vector.

Las Máquinas de Soporte Vectorial (Support Vector Machines SVMs) son un conjunto de algoritmos de aprendizaje supervisados que desarrollan métodos relacionados con los problemas de clasificación y regresión. Como en la mayoría de los métodos de clasificación supervisada, los datos de entrada (los puntos) son vistos como un vector p -dimensional (una lista de p números). Dado un conjunto de puntos como un subconjunto de un conjunto mayor (espacio), en el que cada uno de ellos pertenece a una de dos posibles categorías, de manera que un algoritmo basado en SVM construye un modelo capaz de predecir si un punto nuevo (cuya categoría desconocemos) pertenece a una categoría o a la otra. La SVM, intuitivamente, es un modelo que, partiendo de un conjunto de ejemplos de entrenamiento, podemos etiquetarlos en diferentes clases y representar dichas muestras en puntos en el espacio para tratar de separar las diferentes clases mediante un espacio lo más amplio posible, para que cuando las nuevas muestras de los casos de test se pongan

en correspondencia con dicho modelo puedan ser clasificadas correctamente en función de su proximidad.

En ese concepto de “separación óptima” es donde reside la característica fundamental de las SVM: este tipo de algoritmos buscan el hiperplano que tenga la máxima distancia (margen) con los puntos que estén más cerca de él mismo. Por eso también a veces se les conoce a las SVM como clasificadores de margen máximo. De esta forma, los puntos del vector que son etiquetados con una categoría estarán a un lado del hiperplano y los casos que se encuentren en la otra categoría estarán al otro lado.

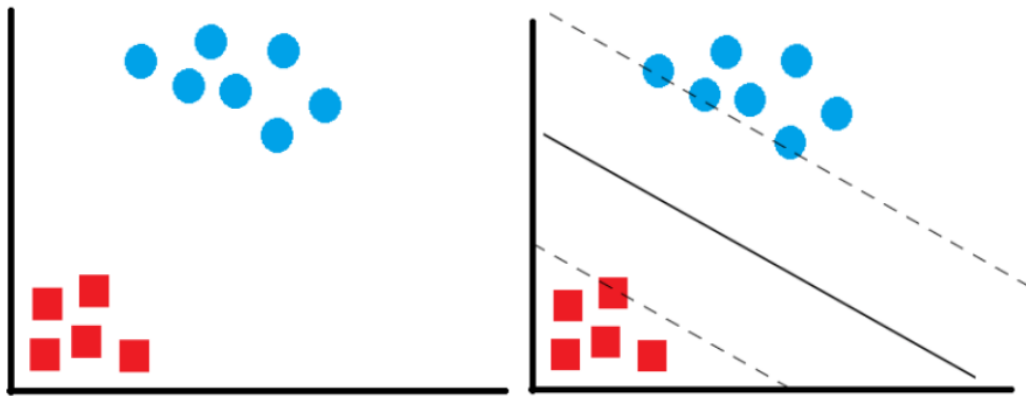


Figura 2. Máquinas soporte vector (Parra, 2019).

Red Neuronal Artificial

Una Red Neuronal Artificial (RNA) es un modelo matemático inspirado en el comportamiento biológico de las neuronas y en cómo se organizan formando la estructura del cerebro. Las redes neuronales intentan aprender mediante ensayos repetidos como organizarse mejor a sí mismas para conseguir maximizar la predicción. La Inteligencia Artificial y el Deep Learning han puesto muy de moda esta técnica tan sofisticada de reconocimiento de patrones que imita las neuronas del cerebro humano ya que es capaz de modelar relaciones extremadamente complejas y suele utilizarse cuando no se conoce la naturaleza exacta de la relación entre los valores de entrada y los de salida (Keyrus, 2022). Las redes neuronales son técnicas de modelado no lineal sofisticadas que son capaces de modelar funciones complejas.

Pueden aplicarse a problemas de predicción, clasificación o control en un amplio espectro de campos como las finanzas, la psicología cognitiva/neurociencia, la medicina, la ingeniería y la física.

Las redes neuronales se utilizan cuando no se conoce la naturaleza exacta de la relación entre los valores de entrada y de salida. Una característica clave de las redes neuronales es que aprenden la relación entre los valores de entrada y salida a través del entrenamiento. Existen tres tipos de entrenamiento en redes neuronales utilizadas por diferentes redes, el aprendizaje por refuerzo, el supervisado y no supervisado, siendo el supervisado el más común (Espino, 2017). Las redes neuronales se hicieron operativas por primera vez a finales de los 50. Rosenblatt F. (1958) creó el perceptrón, un algoritmo de reconocimiento de patrones basado en una red de aprendizaje de computadora de dos capas usando una simple suma y la resta. A finales de los 60 el proceso de investigación sufrió un estancamiento, y ha sido a partir de los años 80 del siglo pasado, cuando se produjo el mayor desarrollo teórico. Son muchos los modelos de redes neuronales, el más utilizado es el algoritmo backpropagation fue creado por Werbos P. (1990).

Tipos de modelos de redes neuronales: Existen actualmente más de 40 paradigmas de redes neuronales artificiales. Se estima que tan sólo cuatro arquitecturas: el modelo perceptrón multicapa (MLP), los mapas auto organizados de Kohonen, (SOFM), el vector de cuantificación (LVQ) y las redes de base radial (RBF) cubren, aproximadamente, el 90% de las aplicaciones prácticas de redes neuronales.



Recuerde que: El modelo más utilizado es el perceptrón multicapa, que abarca el 70% de las aplicaciones, dado que se ha demostrado que este modelo es un aproximador universal de funciones.

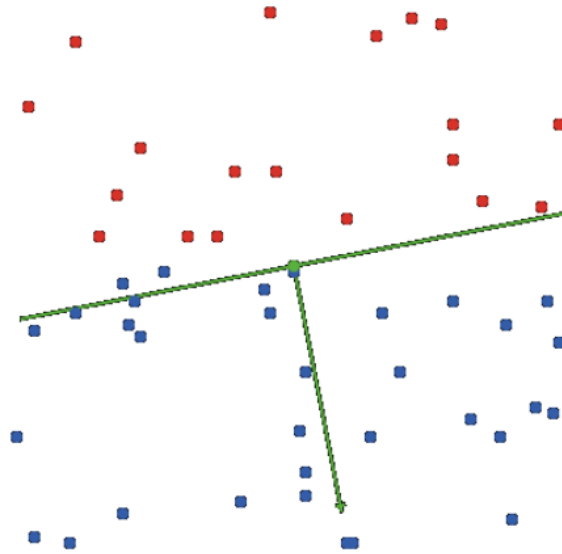


Figura 2. Red Neuronal.

Clasificador Bayesiano

Naïve Bayes es uno de los clasificadores más utilizados por su simplicidad y rapidez. Se trata de una técnica de clasificación y predicción supervisada que construye modelos que predicen la probabilidad de posibles resultados, en base al Teorema de Bayes, también conocido como teorema de la probabilidad condicionada.

Series Temporales y Data Mining

Este método combina una mezcla de técnicas de data mining tradicional como clustering y árboles de decisión, con otras de forecasting con el fin de mejorar las predicciones sobre datos recopilados como ventas por meses o trimestres, llamadas por día, o visitas a nuestra web por hora.

Potenciación del Gradiente

Lleva a cabo un resampling (método de remuestreo) de nuestro dataset para generar unos resultados que formen una media ponderada del conjunto de datos.

Ensemble Models

Es famoso por su precisión debido a la disponibilidad de algoritmos de boosting y bagging. Ensemble models, crea un nuevo modelo entrenando varios modelos

similares combinando los resultados para mejorar la precisión, reducir la varianza y los sesgos e identificar el mejor modelo para usar con nuevos datos.

Modelos de Respuesta Incremental

Suele utilizarse para reducir el Churn o comprobar la efectividad de diferentes acciones de Marketing. Se modela el cambio de probabilidad causado por una acción (Keyrus, 2022).

1.3 Algoritmos de regresión

Regresión Logística

Las regresiones logísticas son utilizadas para predecir el resultado de una variable categórica (una variable que puede adoptar un número limitado de categorías) en función de las variables independientes o predictivas. Es útil para modelar la probabilidad de un evento ocurriendo como función de otros factores. Por ejemplo, puede utilizarse para predecir el riesgo crediticio.

Regresión Lineal

La regresión lineal consiste en una línea recta que muestra el “mejor encaje” de todos los puntos de los valores numéricos. También se llama el método de los mínimos cuadrados porque calcula la suma de las distancias al cuadrado entre los puntos que representan los datos y los puntos de la línea que genera el modelo. Así, la mejor estimación será la que minimice estas distancias.



Recuerde que: Cuando la variable sea discreta los llamaremos clasificación. Como ejemplo se puede clasificar en dos clases (gripe o no) o varias clases (producto “x” a recomendar). Cuando la variable sea continua los llamaremos regresión. La predicción de los precios de una casa dadas las características de la casa como tamaño, precio, etc. es uno de los ejemplos comunes de regresión (Paredes, 2020).

Preparar los datos para el modelo de aprendizaje

Antes de comenzar a usar modelos de pronóstico supervisados tenemos que preparar los datos de origen y realizar unas tareas que nos den el control del proceso y nos permitan verificar, al finalizar, la bondad y ajuste de nuestras previsiones. Es una buena práctica y debemos empezar siempre por ella, dividir el conjunto de datos de origen en dos subconjuntos, uno de ellos nos servirá para el entrenamiento (*train*) del modelo y el otro para la comprobación -a posteriori- (*test*). Con este sistema de división de la muestra inicial de datos evitamos el *overfitting* tan preocupante en los modelos de predicción. Otro de los trabajos preparatorios de importancia es la factorización o agrupamiento de los datos. En casos como el modelo de *naive bayes*, el uso de valores numéricos continuos en los datos de origen genera una sobredimensión exponencial de los casos posibles, por exceso de combinatoria entre variables que desborda el modelo y puede producir errores en los pronósticos.

En estos casos es siempre recomendable factorizar, agrupar y disminuir la dimensión origen de las variables de entrada agrupando valores. En otros casos como el algoritmo *knn*, la medición de distancia entre variables es el eje del pronóstico, por lo que es necesario escalar los datos normalizarlos para equiparar la fórmula de distancia entre las variables, como veremos en los ejemplos.

Crear particiones de la muestra

El primer paso en todo análisis debe ser dividir la muestra de datos en dos conjuntos de datos uno para entrenamiento y otro para test. Esto se puede hacer a mano, por ejemplo, usando la función `sample`, o con la ayuda de algunos paquetes que llevan funciones incorporadas para las particiones de datos.

Ejemplo de partición a mano

Se puede usar la base de datos de muestra de supervivientes del Titanic que se da como tabla en dataset. Para ver todas las series y bases de datos disponibles en dataset escribiremos `data()`. Titanic es una tabla que indica casos y frecuencias de cada caso, por lo que para crear la tabla completa hay que expandir la tabla origen, y repetir cada caso el número de veces que indica la columna frecuencia.

```

# cargamos los datos
data("Titanic")
str(Titanic)
## 'table' num [1:4, 1:2, 1:2, 1:2] 0 0 35 0 0 0 17 0 118 154 ...
## - attr(*, "dimnames")=List of 4
## ..$ Class : chr [1:4] "1st" "2nd" "3rd" "Crew"
## ..$ Sex : chr [1:2] "Male" "Female"
## ..$ Age : chr [1:2] "Child" "Adult"
## ..$ Survived: chr [1:2] "No" "Yes"

class(Titanic)
## [1] "table"

# Transformamos los datos en una dataframe

Titanic_df=as.data.frame(Titanic)
str(Titanic_df)
## 'data.frame': 32 obs. of 5 variables:
## $ Class : Factor w/ 4 levels "1st","2nd","3rd",...: 1 2 3 4 1 2 3 4 1 2
## ...
## $ Sex : Factor w/ 2 levels "Male","Female": 1 1 1 1 2 2 2 2 1 1 ...
## $ Age : Factor w/ 2 levels "Child","Adult": 1 1 1 1 1 1 1 1 2 2 ...
## $ Survived: Factor w/ 2 levels "No","Yes": 1 1 1 1 1 1 1 1 1 1 ...
## $ Freq : num 0 0 35 0 0 0 17 0 118 154 ...

# Creamos una tabla de casos completos a partir de la frecuencia de cada uno
# Esto repite cada caso el num de veces que se ha dado
# según la frecuencia que está en la columna Freq de la tabla.
repetir_secuencia=rep.int(seq_len(nrow(Titanic_df)), Titanic_df$Freq)
# tenemos una serie con el numero de registro de la tabla original y las veces
# que se repite

# Creamos una nueva tabla repitiendo los casos según el modelo anterior.

Titanic_data=Titanic_df[repetir_secuencia,]

# Ya no necesitamos la columna de frecuencias y la borramos.

Titanic_data$Freq=NULL
head(Titanic_data)
## Class Sex Age Survived
## 3 3rd Male Child No
## 3.1 3rd Male Child No
## 3.2 3rd Male Child No
## 3.3 3rd Male Child No
## 3.4 3rd Male Child No
## 3.5 3rd Male Child No

# Como vemos todo son factores

str(Titanic_data)
## 'data.frame': 2201 obs. of 4 variables:
## $ Class : Factor w/ 4 levels "1st","2nd","3rd",...: 3 3 3 3 3 3 3 3 3 3
## ...
## $ Sex : Factor w/ 2 levels "Male","Female": 1 1 1 1 1 1 1 1 1 1 ...
## $ Age : Factor w/ 2 levels "Child","Adult": 1 1 1 1 1 1 1 1 1 1 ...
## $ Survived: Factor w/ 2 levels "No","Yes": 1 1 1 1 1 1 1 1 1 1 ...

```

En este caso buscamos dividir la tabla en dos conjuntos uno de entrenamiento con el 75% de los registros y otro de comprobación o de test con el 25% restante de los registros que nos servirá para validar el modelo.

```
# -----
# DIVISION A MANO
# contamos el num de registro de la base de datos del titanic
nrow(Titanic_data)
## [1] 2201
# calculamos el 75%
num_reg_entrenamiento<-as.integer(0.75*nrow(Titanic_data))
num_reg_entrenamiento
## [1] 1650
# Creamos una vector de 75% de los registros aleatorio
titanic_train <- sample(nrow(Titanic_data), num_reg_entrenamiento)
# Creamos el conjunto de registros de entrenamiento pasando ese vector a la
tabla
d_titanic_train <- Titanic_data[titanic_train,]
head(d_titanic_train)
##      Class Sex Age Survived
## 12.53  Crew Male Adult      No
## 28.107 Crew Male Adult     Yes
## 12.115 Crew Male Adult      No
## 12.94   Crew Male Adult      No
## 10.144 2nd  Male Adult      No
## 29.44   1st Female Adult     Yes
# Creamos los datos de comprobación o test (notese el -)
d_titanic_test <- Titanic_data[-titanic_train,]
head(d_titanic_test)
##      Class Sex Age Survived
## 3      3rd Male Child      No
## 3.7    3rd Male Child      No
## 3.8    3rd Male Child      No
## 3.13   3rd Male Child      No
## 3.25   3rd Male Child      No
## 3.28   3rd Male Child      No
```

Usando una formulación simple hemos creado dos conjuntos de muestra aleatorios excluyentes de entrenamiento y muestra.

Ejemplo de partición con library(caret)

Veamos otra forma de hacerlo usando la librería library(caret)

```
library(caret)
set.seed(987654321)
# creamos un vector de particion sobre la variable Survived
# el tamaño de muestra será de 75%
trainIndex=createDataPartition(Titanic_data$Survived, p=0.75)$Resample1
d_titanic_train=Titanic_data[trainIndex, ]
d_titanic_test= Titanic_data[-trainIndex, ]
```


Categorización de los datos origen

Existe un problema en el uso de las funciones de clasificación cuando las combinaciones posibles entre variables tienden a ser infinitas. Esto sucede cuando, por ejemplo, una de las variables es de tipo numérico, y tiene datos continuos. La mayoría de los modelos de clasificación solo son capaces de trabajar un número limitado de categorías, y, por lo tanto, es necesario agrupar los datos originales y reducir las opciones combinatorias, por lo que hay que evitar siempre el uso de variables continuas en los datos. Si no realizamos la reducción de categorías nos arriesgamos a obtener errores, incluso evidentes, en los pronósticos. Un caso evidente es el uso de la función de `naive_bayes` que maneja muy mal los datos continuos, pues está pensado para variables categorizadas. La solución es realizar una categorización previa de los datos que evite el problema y a la vez simplifique el modelo de pronóstico. Categorizar significa agrupar los datos de las variables continuas en categorías próximas, simplificando las salidas y reduciendo las combinaciones. Un ejemplo claro es redondear las salidas numéricas a números divisibles por 10 o por 5, o sustituir la variable por los cuantiles más representativos. Para transformar las variables y categorizarlas podemos usar varias funciones de R como:

- convertir a factor con la función `as.factor()`. * Las categorías de un factor se ven con la función `levels()` * los nombres de esas categorías los damos con la función `lables()`
- la función `table(tabla_1$hora)` cuenta y resumen los datos
- las funciones `quantile()` o `cut()` ayudan a dividir y categorizar variables continuas
- funciones de redondeo

Vamos a ver varios ejemplos con la dataset “women” que contiene las alturas y pesos de mujeres americanas de edad entre 30 y 39 años. Ambos datos de altura y peso son continuos y toman 15 posibles valores, por lo que las combinaciones cruzadas dan muchísimos casos posibles. Para simplificar las combinatorias vamos a categorizar los datos de varias maneras, como ejemplos:

```
# cargamos los datos de
data("women")
str(women)
```

```
## 'data.frame': 15 obs. of 2 variables:
## $ height: num 58 59 60 61 62 63 64 65 66 67 ...
## $ weight: num 115 117 120 123 126 129 132 135 139 142 ...
```

```
summary(women)
```

```
##      height      weight
## Min.   :58.0   Min.   :115.0
## 1st Qu.:61.5   1st Qu.:124.5
## Median :65.0   Median :135.0
## Mean   :65.0   Mean   :136.7
## 3rd Qu.:68.5   3rd Qu.:148.0
## Max.   :72.0   Max.   :164.0
```

```
length(unique(women$height)) # numero de registros unicos
```

```
## [1] 15
```

```
length(unique(women$weight))
```

```
## [1] 15
```

```
# opción 1. creamos una función de redondeo
```

```
redondea5<-function(x,base=5){
  as.integer(base*round(x/base))
}
```

```
# Copiamos la tabla con el nombre nuevo
```

```
mujeres_a<-women
```

```
# pasamos los datos a sistema internacional
```

```
mujeres_a$peso<-mujeres_a$weight*0.453592 # paso a kg
mujeres_a$peso<- redondea5(mujeres_a$peso,10)
length(unique(mujeres_a$peso))
```

```
## [1] 3
```

```
mujeres_a$altura<-mujeres_a$height*2.54 # paso a cm
mujeres_a$altura<- redondea5(mujeres_a$altura,10)
length(unique(mujeres_a$altura))
```

```
## [1] 4
```

```
head(mujeres_a)
```

```
##   height weight peso altura
## 1     58    115   50    150
## 2     59    117   50    150
## 3     60    120   50    150
## 4     61    123   60    150
## 5     62    126   60    160
## 6     63    129   60    160
```

Con la simplificación anterior hemos pasado de 15x15 casos combinatorios a solo 3x4. Podríamos usar también factores para convertir los datos

```
#Ejemplo de transformación a factor
mujeres_a$peso1<- factor(mujeres_a$peso,
                        levels = c(50, 60, 70),
                        labels = c( "flaca","media","gordita"))

# Ejemplo 2 de trans a factor:
mujeres_a$altura1 <- factor(mujeres_a$altura, levels = c(150,160,170,180),
labels = c("Bajo","Medio","Alta", "muy alta"))
head(mujeres_a)
```

```
## height weight peso altura peso1 altura1
## 1 58 115 50 150 flaca Bajo
## 2 59 117 50 150 flaca Bajo
## 3 60 120 50 150 flaca Bajo
## 4 61 123 60 150 media Bajo
## 5 62 126 60 160 media Medio
## 6 63 129 60 160 media Medio
```

Manejo de NA

En todos los modelos, la existencia de registros con falta de datos o NA, anula el valor de dicha evidencia en el modelo de entrenamiento. Una solución es completar estos casos con las funciones como `impute()` del paquete `e1071` que sustituye el NA por un valor estimado, que puede ser la media. En la tabla de ejemplo `donors` hay muchos datos de la edad de los clientes que faltan.

```
# Vemos cuantos datos de edad faltan.
set.seed(333)
datos<-data.frame(a=sample(1:10, 100,replace = T))
datos$a[c(1,3)] <- NA
head(datos)
```

```
## a
## 1 NA
## 2 1
## 3 NA
## 4 6
## 5 1
## 6 8
```

```
library(e1071)
# Imputamos los nuevos datos estimados de edad asignando usando impute
datos$imputed <- impute(datos)
head(datos)
## a a
## 1 NA 5
```

```
## 2 1 1
## 3 NA 5
## 4 6 6
## 5 1 1
## 6 8 8
```

```
# Otra forma es hacerlo manualmente
datos$imputed2<-ifelse(is.na(datos$a),5,datos$a)
head(datos)
##      a a imputed2
## 1 NA 5          5
## 2 1 1          1
## 3 NA 5          5
## 4 6 6          6
## 5 1 1          1
## 6 8 8          8
```

Tema 2: Técnicas de aprendizaje supervisado: clasificación y regresión.

2.1 Clasificación con Naive Bayes (Bayesiano ingenuo)

Naive Bayes es un modelo de predicción basado en la probabilidad Bayesiana. El modelo es muy simple, pero poderoso, en cuanto que es resultado directo de los datos y su tratamiento con simple estadística bayesiana de la probabilidad condicionada. Hay que tener en cuenta que se asume, por simplificación que las variables son todas sucesos independientes. La función de clasificación ingenua de bayes se encuentra en varias librerías de R en: **naivebayes**, en el paquete **e1071** y en otros (Villalba, 2018b).

El modelo bayesiano de probabilidad condicionada se representa como: $P(A|B)=P(A \cap B)/P(B)$ $P(A|B)=P(A \cap B)/P(B)$. Es decir, la probabilidad de que se dé el caso AA dado BB es igual a la probabilidad de la intersección de A con B ($A \cap B$) partido la probabilidad de BB. Estirando esta formulación llegaríamos al teorema de Bayes cuya expresión más típica es la siguiente: $P(A|B)=P(B|A)*P(A)/P(B)$.

2.1.1 Crear un modelo con naivebayes

La tabla que vamos a crear contiene 3 variables: la hora del día, el lugar donde está Juan a esa hora, y otra columna que nos indica si es o no fin de semana con un valor lógico (TRUE o FALSE). Vamos a crear la tabla para el ejemplo:

```
# leemos la base de datos
# tabla_1<-read.csv("tabla_1.csv",header = TRUE)

# vamos a crear el ejemplo de cero: CREAMOS UNA TABLA DE DATOS
tabla_1<-
data.frame(hora=c(8,14,24,8,14,24,8,14,24,8,14,24,8,14,24,24,24))
tabla_1$lugar<-c("casa","restaurante","casa",
                 "trabajo","trabajo","casa",
                 "trabajo","trabajo","casa",
                 "casa","restaurante","casa",
                 "trabajo","trabajo","casa",
                 "casa","restaurante","casa","cine","cine")
tabla_1$finde<-c(T,T,T,
                 F,F,F,
                 F,F,F,
                 T,T,T,
                 F,F,F,
                 T,T,T,
                 T,F,
                 )

str(tabla_1)
## 'data.frame':    20 obs. of  3 variables:
## $ hora : num  8 14 24 8 14 24 8 14 24 8 ...
## $ lugar: chr  "casa" "restaurante" "casa" "trabajo" ...
## $ finde: logi  TRUE TRUE TRUE FALSE FALSE FALSE ...

head(tabla_1)
##   hora lugar finde
## 1    8   casa  TRUE
## 2   14 restaurante TRUE
## 3   24   casa  TRUE
## 4    8   trabajo FALSE
## 5   14   trabajo FALSE
## 6   24   casa  FALSE

# vemos como ejemplo el número de registros de hora según el lugar
table(tabla_1$hora,tabla_1$lugar)
##
##      casa cine restaurante trabajo
##    8      3      0          0      3
##   14      0      0          3      3
##   24      6      2          0      0
```

Como vemos, es una tabla con 20 registros y 3 variables en columnas, sobre la que queremos practicar pronósticos bayesianos de probabilidad condicionada. Vamos cargar la librería `naivebayes` con objeto de crear un modelo de pronóstico de

la variable dependiente lugar a partir de las variables independientes hora y finde. Este modelo nos diría por ejemplo la probabilidad de que: sabiendo la hora y si es o no fin de semana, Juan se encuentre en un lugar determinado.

Como vimos en las consideraciones previas, los modelos de pronóstico bayesiano, y en particular naivebayes funcionan muy mal con datos numéricos continuos, y vamos a ver la prueba, pues crearemos un modelo con la variable hora tal cual, y despues haremos el mismo modelo con la variable hora convertida en factor.

Primero creamos la fórmula de modelo con la función `naive_bayes()` y luego definimos un hecho, una ocurrencia concreta de los parámetros y llamamos a la función `predict()`. Esta función es común a la mayoría de los modelos de predictivos, y sus argumentos son el nombre del modelo y un hecho almacenado como `data.frame`. Si añadimos el argumento `type="prob` nos da el resultado como pronóstico probabilístico y si no, solo el pronóstico más probable.

```
# cargamos la librería
library(naivebayes)
# creamos el modelo de pronóstico
m <- naive_bayes(lugar ~ hora+finde, data = tabla_1)#, laplace = 1)

# representamos graficamente el modelo
plot(m)
```

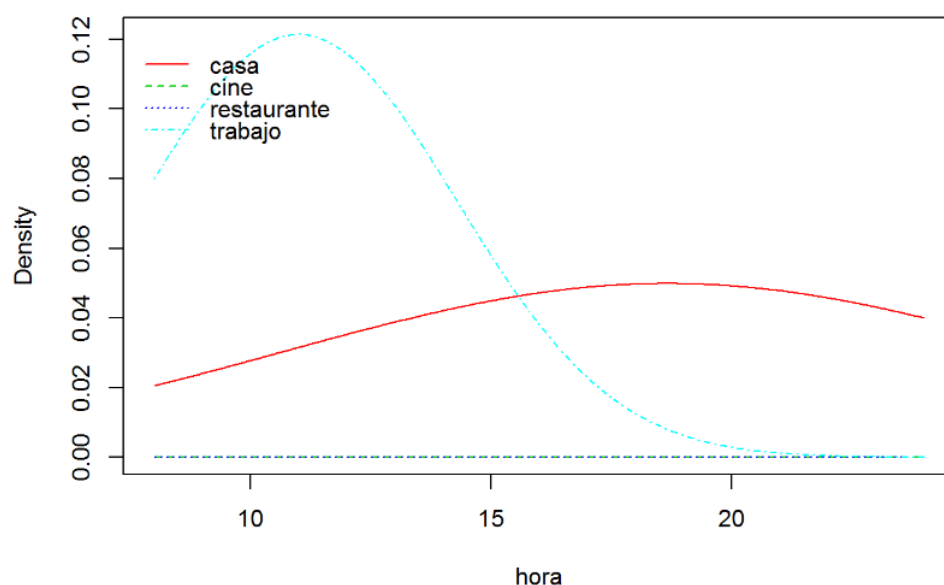


Figura 3a. Representación gráfica del modelo.

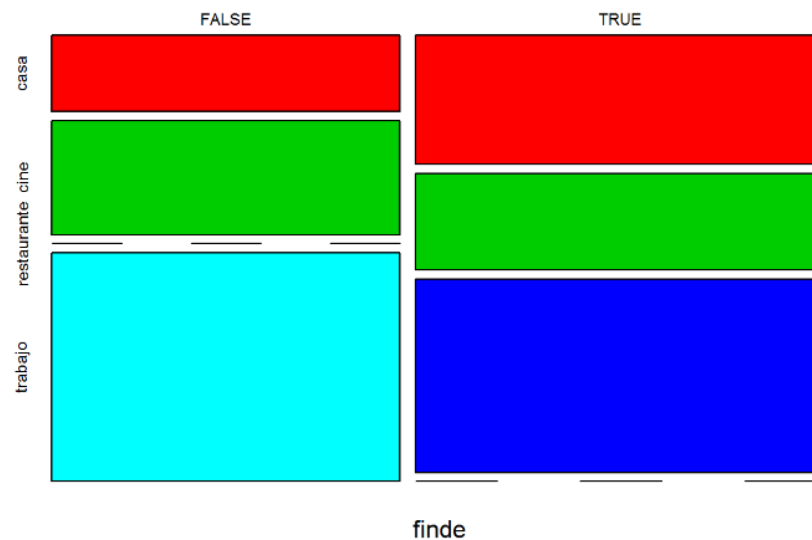


Figura 3b. Representación gráfica del modelo.

```
# ejecutando predict(modelo) tenemos los resultados de pronostico para cada
registro de datos
tabla_1$p=predict(m)
head(tabla_1)

##   hora      lugar finde      p
## 1    8        casa  TRUE     casa
## 2   14 restaurante  TRUE restaurante
## 3   24         casa  TRUE      cine
## 4    8     trabajo FALSE   trabajo
## 5   14     trabajo FALSE   trabajo
## 6   24         casa FALSE      cine

# pero si queremos un hecho concreto:
# creamos un hecho a priori, sobre el que queremos pronosticar el resultado
# como el modelo es lugar ~ hora+finde, aportamos un dato de hora y otro de
finde
# en este caso queremos pronosticar donde se encuentra Juan a las 14 horas un
día laborable

h<-data.frame(hora= 24, finde=T)
table(tabla_1$lugar,tabla_1$hora+tabla_1$finde)

##
##           8 9 14 15 24 25
## casa      0 3 0 0 3 3
## cine      0 0 0 0 1 1
## restaurante 0 0 0 3 0 0
## trabajo   3 0 3 0 0 0

# Llamamos a la función de predicción
predict(m,h)

## [1] cine
## Levels: casa cine restaurante trabajo
##           casa      cine restaurante trabajo
## [1,] 0.0006001881 0.9993923 7.515315e-06      NaN
```

La predicción que obtenemos con el modelo para (*hora= 24, finde=T*) es claramente errónea, pues solo 1 de los 4 registros que tenemos a las 24 horas en fin de semana es ir al cine, los otros 3 son estar en casa, por lo que algo falla en el modelo al ser el evento más probable estar en casa.

Este problema es habitual cuando usamos datos continuos, que nos generan distribuciones de probabilidad continuas. En este caso el evento de ir al cine tiene muy pocos datos, pero siempre a las 24 horas, por lo que la media se mantiene en 24 h. Sin embargo, el hecho estar en casa tienen muchos registros en diferentes horas, por lo que el valor medio de la hora es un número intermedio 18,6.

Para evitar problemas debemos transformar las variables continuas en discretas y reducir al máximo los valores posibles realizando lo que denominamos una categorización previa de los datos. Por ejemplo, convirtiendo los datos en factores.

```
# Convertimos la variable continua numérica hora, en factor discreto
tabla_1$hora<-as.factor(tabla_1$hora)
str(tabla_1)

## 'data.frame':    20 obs. of  4 variables:
## $ hora : Factor w/ 3 levels "8","14","24": 1 2 3 1 2 3 1 2 3 1 ...
## $ lugar: chr  "casa" "restaurante" "casa" "trabajo" ...
## $ finde: logi  TRUE TRUE TRUE FALSE FALSE FALSE ...
## $ p     : Factor w/ 4 levels "casa","cine",...: 1 3 2 4 4 2 4 4 2 1 ...

# calculamos de nuevo el modelo ahora
m <- naive_bayes(lugar ~ hora+finde, data = tabla_1)
# Hacemos de nuevo la predicción
predict(m,h)

## [1] casa
## Levels: casa cine restaurante trabajo

predict(m,h, type="prob")

##          casa          cine  restaurante  trabajo
## [1,] 0.8571429 0.1428571 1.432592e-123      NaN
# ojo al crear el hecho que debe ser acorde a los datos,
# si es factor debe contener en levels los mismos que la tabla origen
# por ello lo creamos a partir de esta tabla mejor

h<-tabla_1[1,c(1,3)]
h$hora="24"
h$finde=F
predict(m,h)
## [1] casa
## Levels: casa cine restaurante trabajo
```


Como hemos visto al transformar en factor la variable numérica continua, hemos realizado un pronóstico más acorde con los datos.

2.1.2 Corrección de laplace

En muchas ocasiones los datos no contienen muestras a priori de todas las combinaciones de variables posibles, por lo que las probabilidades de casos raros salen excesivamente bajas. Para corregir esto el modelo `naiveBayes` tiene la opción de añadir en la fórmula el argumento de `laplace=1`, en el que indicamos que, al menos, se debe contar con una aparición de cada posible combinación de factores. Este parámetro se puede aumentar a criterio del investigador, y permite incorporar casos raros dentro del pronóstico que, de otra forma, por la simplificación del modelo de Bayes, darían probabilidad cero.

Por ejemplo, en los datos de Juan y la ubicación según las horas, no tenemos ningún registro de que vaya a trabajar en fin de semana, pero eso no significa que no tengamos cierta probabilidad, lo que se podría solventar añadiendo al modelo `laplace=1`. Veamos los cambios al reformular el modelo con `laplace`. Gráficamente se aprecia que, por ejemplo, la probabilidad de ir a restaurante entre semana pasa de cero a una cantidad pequeña, y de trabajar el fin de semana igual (pasa de cero a una proporción).

```
# cargamos la librería
require(naivebayes)

# Modelo de pronóstico sin Laplace
m <- naive_bayes(lugar ~ hora+finde, data = tabla_1)
plot(m)
```

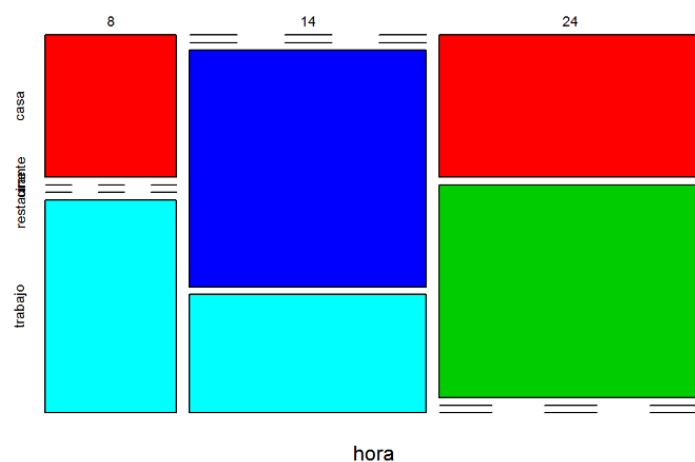


Figura 4a. Representación gráfica del modelo.

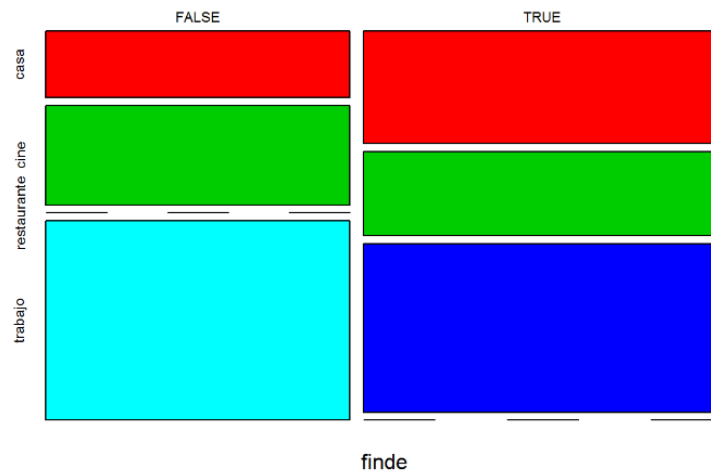


Figura 4b. Representación gráfica del modelo.

```
# cambiamos el modelo añadiendo Laplace con al menos una ocurrencia por evento
m1 <- naive_bayes(lugar ~ hora+finde, data = tabla_1, laplace=1)
plot(m1)
```

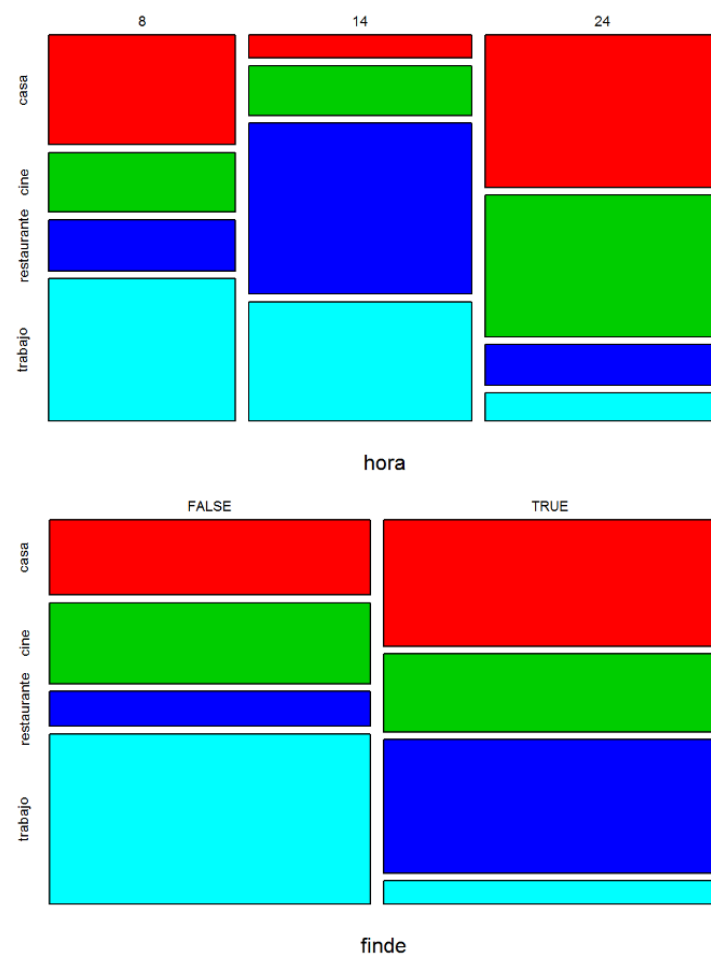


Figura 5. Representación gráfica del modelo.

```

# ¿dónde está Juan si son las 14 horas en fin de semana?
h$hora=8
h$finde=T
predict(m,h,type="prob")

##           casa      cine restaurante trabajo
## [1,] 0.8571429 0.1428571 2.165259e-12      NaN
predict(m1,h,type="prob")
##           casa      cine restaurante      trabajo
## [1,] 0.8790698 0.1209302 1.91539e-39 4.420521e-77

```

2.1.3 Probando el paquete e1071

Se probará otro paquete que contiene a naive_Bayes el `e1071`. Usaremos los datos de supervivientes del Titanic que vienen en los datasets de R por defecto. La tabla de datos tiene 32 filas pero en realidad esconde en la columna `freq` el número de repeticiones de cada caso, por lo que el primer paso es crear una tabla completa.

```

library(e1071)
#Cargamos los datos del Titanic desde datasets
data("Titanic")
#Los almacenamos en un data frame
Titanic_df=as.data.frame(Titanic)
str(Titanic_df)

## 'data.frame':    32 obs. of  5 variables:
## $ Class      : Factor w/ 4 levels "1st","2nd","3rd",...: 1 2 3 4 1 2 3 4 1 2
## ...
## $ Sex        : Factor w/ 2 levels "Male","Female": 1 1 1 1 2 2 2 2 1 1 ...
## $ Age        : Factor w/ 2 levels "Child","Adult": 1 1 1 1 1 1 1 1 2 2 ...
## $ Survived   : Factor w/ 2 levels "No","Yes": 1 1 1 1 1 1 1 1 1 1 ...
## $ Freq       : num  0 0 35 0 0 0 17 0 118 154 ...

#Creamos una tabla de casos completos a partir de la frecuencia de cada uno
repeating_sequence=rep.int(seq_len(nrow(Titanic_df)), Titanic_df$Freq)
#Esto repite cada caso según la frecuencia dada en la col de la tabla.

#Creamos una nueva tabla repitiendo los casos según el modelo anterior.
Titanic_dataset=Titanic_df[repeating_sequence,]
#Ya no necesitamos la tabla de frecuencias más.
Titanic_dataset$Freq=NULL
head(Titanic_dataset)

##      Class Sex Age Survived
## 3      3rd Male Child      No
## 3.1    3rd Male Child      No
## 3.2    3rd Male Child      No
## 3.3    3rd Male Child      No
## 3.4    3rd Male Child      No
## 3.5    3rd Male Child      No

# todos son factores
str(Titanic_dataset)

```

```
## 'data.frame': 2201 obs. of 4 variables:
## $ Class : Factor w/ 4 levels "1st","2nd","3rd",...: 3 3 3 3 3 3 3 3 3 3 ...
## $ Sex : Factor w/ 2 levels "Male","Female": 1 1 1 1 1 1 1 1 1 1 ...
## $ Age : Factor w/ 2 levels "Child","Adult": 1 1 1 1 1 1 1 1 1 1 ...
## $ Survived: Factor w/ 2 levels "No","Yes": 1 1 1 1 1 1 1 1 1 1 ...

# Ajustamos un modelo de naive bayes con La Librería e1071
m.e1071 <- naiveBayes(Survived ~ ., data = Titanic_dataset)
m.e1071 # vemos el modelo

##
## Naive Bayes Classifier for Discrete Predictors
##
## Call:
## naiveBayes.default(x = X, y = Y, laplace = laplace)
##
## A-priori probabilities:
## Y
## No Yes
## 0.676965 0.323035
##
## Conditional probabilities:
## Class
## Y 1st 2nd 3rd Crew
## No 0.08187919 0.11208054 0.35436242 0.45167785
## Yes 0.28551336 0.16596343 0.25035162 0.29817159
##
## Sex
## Y Male Female
## No 0.91543624 0.08456376
## Yes 0.51617440 0.48382560
##
## Age
## Y Child Adult
## No 0.03489933 0.96510067
## Yes 0.08016878 0.91983122

# realizamos la predicción con el modelo
predicciones.m<-predict(m.e1071,Titanic_dataset)

# Matriz de confusión
table(predicciones.m,Titanic_dataset$Survived)

##
## predicciones.m No Yes
## No 1364 362
## Yes 126 349

hecho<-data.frame(Class="1rd",Sex="Female",Age="Child")
predict(m.e1071,hecho)

## [1] Yes
## Levels: No Yes
```

2.2 Clasificación con Árbol de Decisión.

Un árbol de decisión es una estructura ramificada que muestra las diferentes opciones y sus consecuencias. Los puntos en los que hay que tomar decisiones se muestran como nodos, las ramas unen estos nodos y las decisiones últimas son las hojas, donde el camino termina (también se denominan nodos terminales). Existen varios paquetes de R que permiten hacer árboles de decisión (Villalba, 2018a).

2.2.1 Librería rpart

Esta librería `rpart` hace árboles de decisión a partir de tablas. La función principal es `rpart()` que crea, a partir de un conjunto de datos, y de una fórmula de predicción, un árbol de decisión que puede usarse para pronosticar con la función `predict`.

Ejemplo 1. Para considerar un ejemplo creamos nuestros propios datos. Tenemos una tabla en la que vienen la altura del padre, de la madre y de un hijo. Y queremos ver su relación.

```
# creamos los datos de ejemplo
# una tabla con alturas del padre la madre, el sexo del hijo y la altura

redondea5<-function(x,base=5){
  as.integer(base*round(x/base))
}

a.padre<- redondea5(rnorm(1000, 168, 25),10)
a.madre<- redondea5(rnorm(1000, 150, 10),10)
s.hijo<-factor(rbinom(1000,1,0.5), levels=c(0,1),labels=c("M","F"))
# creo data.frame
t.alturas<-data.frame(a.padre,a.madre,s.hijo)
# Se calcula la altura del hijo con esta formula
t.alturas$a.hijo<-ifelse(t.alturas$s.hijo == "M",
  (t.alturas$a.padre
+t.alturas$a.madre)*rnorm(1,1,0.07)/2,
  (t.alturas$a.padre
+t.alturas$a.madre)*rnorm(1,1,0.05)/2)
t.alturas$a.hijo<-redondea5(t.alturas$a.hijo,10)
str(t.alturas)

## 'data.frame':    1000 obs. of  4 variables:
## $ a.padre: int   160 210 140 180 200 160 190 130 180 180 ...
## $ a.madre: int   140 160 160 150 160 150 140 150 160 140 ...
## $ s.hijo : Factor w/ 2 levels "M","F": 1 1 2 1 2 1 1 2 1 1 ...
## $ a.hijo : int   160 200 160 180 190 170 180 150 180 170 ...

knitr::kable(head(t.alturas,10), "markdown")
```

a.padre	a.madre	s.hijo	a.hijo
160	140	M	160
210	160	M	200
140	160	F	160
180	150	M	180
200	160	F	190
160	150	M	170
190	140	M	180
130	150	F	150
180	160	M	180
180	140	M	170

```
# Crear un modelo de clasificación con rpart
```

```
library(rpart)
library(rpart.plot)
```

```
# creamos un modelo de clasificación en el que intervengan todas las variables
# el coeficiente cp nos extiende o acorta el arbol, simplifica resultados
```

```
model.alturas1 <- rpart(a.hijo ~ .,
                        data = t.alturas, method = "class", cp = .02)
```

```
# pintamos el modelo
```

```
rpart.plot(model.alturas1, fallen.leaves = FALSE,
            main = "Arbol de decision de alturas hijo\n(en funcion
altura padres)\n",
            shadow.col = "gray")
```

```
## Warning: All boxes will be white (the box.palette argument will be ignored)
because
## the number of classes predicted by the model 10 is greater than
length(box.palette) 6.
## To silence this warning use box.palette=0 or trace=-1.
```

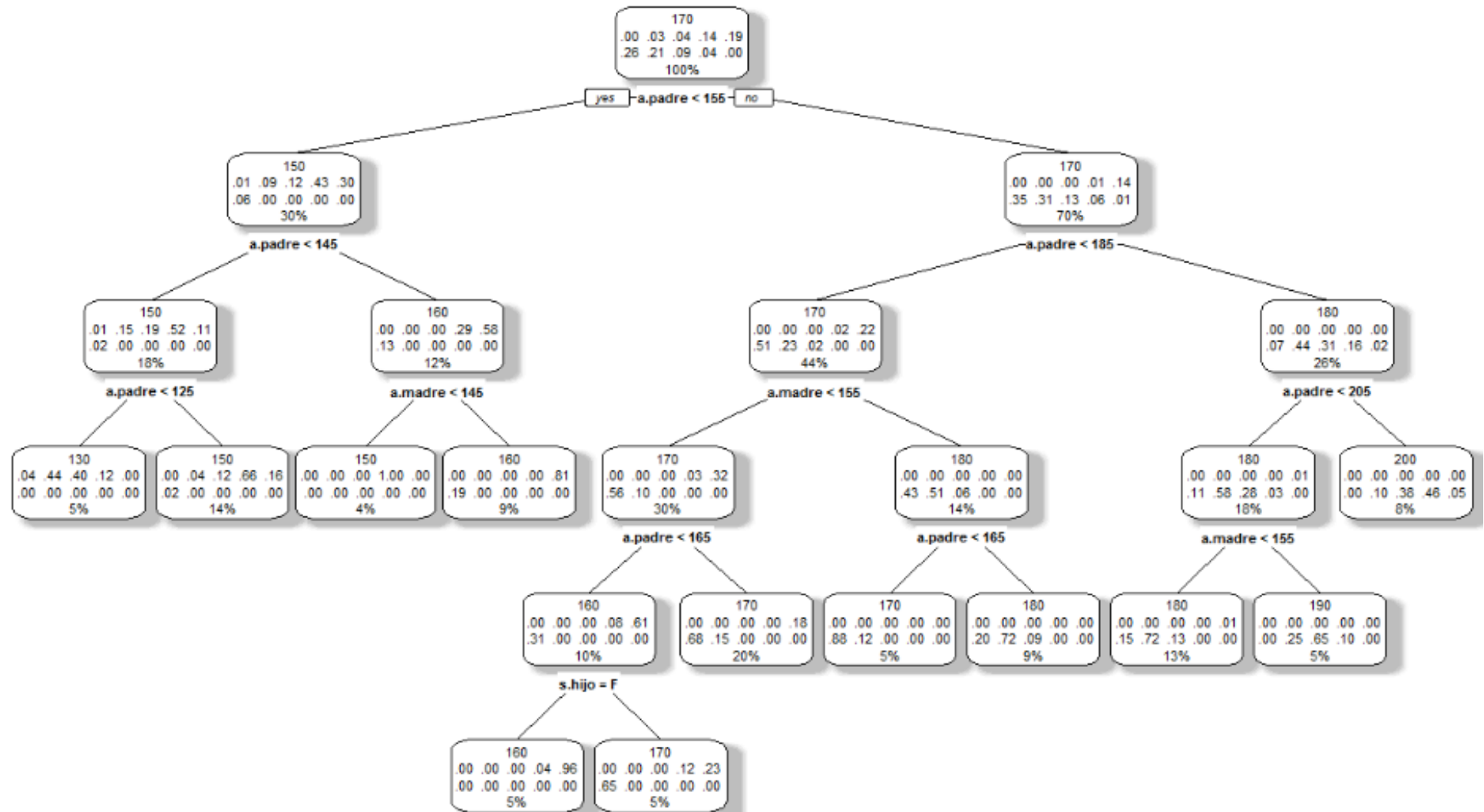


Figura 6. Árbol de decisión de alturas hijo (en función altura padres)

```
# hacemos un pronóstico

h2<-data.frame(a.padre= c(190,150),a.madre= c(180,140),s.hijo=
c("M","M"))
predict(model.alturas1, h2,type = "class")

##    1    2
## 190 150
## Levels: 120 130 140 150 160 170 180 190 200 210
```

Ejemplo 2. Vamos a ver otro ejemplo con los datos del Titanic que trae R por defecto.

```
data(ptitanic)
#-----
sobrevive.model <- rpart(survived ~ ., data = ptitanic, cp = .02)
# cp = .02 for small demo tree
# representamos el modelo

rpart.plot(sobrevive.model,
  main = "Supervivientes del Titanic \n (respuesta binaria)")
```

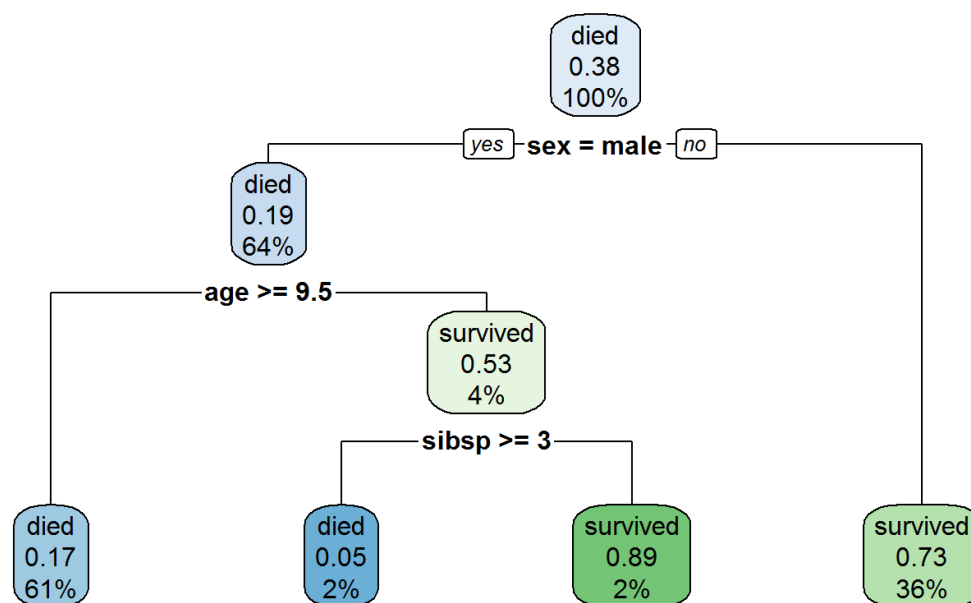


Figura 7. Supervivientes del Titanic (respuesta binaria)

```
# representamos el modelo de otra forma

rpart.plot(sobrevive.model, type = 2, clip.right.labs = FALSE,
  branch = .6,
  box.palette = c( "red","green"), # override default GnBu
palette
  main = "Supervivientes del Titanic \n")
```

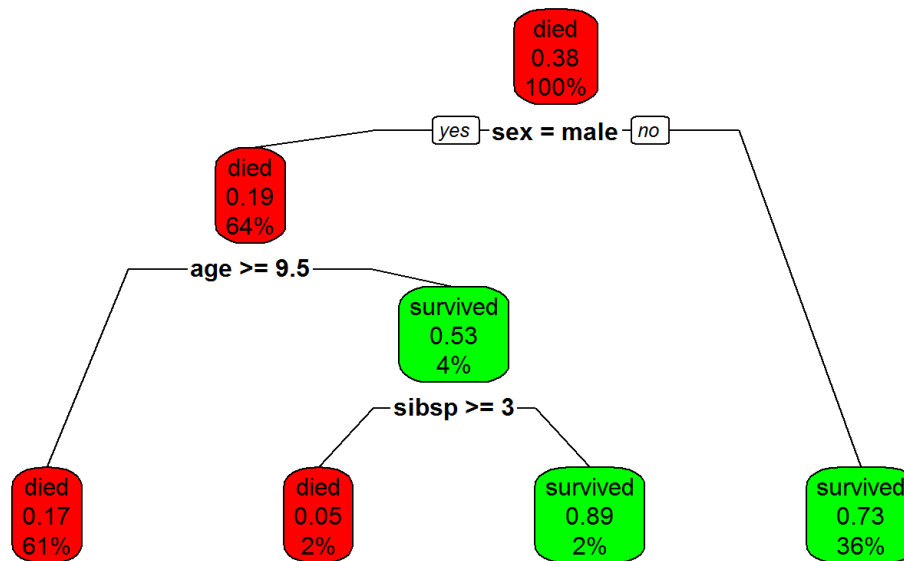



Figura 8. Supervivientes del Titanic.

2.2.2 Overfitting

Los árboles de decisión son bastante problemáticos con la sobrestimación de parámetros, pues la metodología obliga siempre a divisiones paralelas a los ejes de variables y puede generar muchas ramas, pese a que el modelo puede ser muy sencillo. Son necesarias varias acciones previas siempre antes de emprender la clasificación.

1. Simplificar los datos, normalizar o realizar acciones que agrupen los datos y eviten la multiplicación de casos cruzados. Por ejemplo, el simple hecho de redondear los valores numéricos a base 5 o 10, puede eliminar de golpe miles de opciones irrelevantes.
2. Usar los parámetros de `rpart` para simplificar el modelo. Esto se hace con el argumento `rpart.control(cp= 0.2, maxdepth = 30, minsplit = 20)` de varias formas:
 - a. **cp** es un control global que simplifica todo
 - b. **maxdepth** indica el máximo número de ramas
 - c. **minsplit** indica el número mínimo de ocurrencias de ese conjunto para considerarlo en un grupo.
3. Otro aspecto fundamental es que se recomienda dividir los datos de partida en dos conjuntos, uno con el 75% de los registros para entrenamiento y otro con el 25% de los datos para test o comprobación del ajuste del modelo. Esto nos hace simplificar y no sobredimensionar el modelo.

La función **sample()** es muy útil en la tarea de seleccionar una muestra de test y otra de entrenamiento.

```
# ejemplo de división de una muestra
# contamos el num de registro de la base de datos del titanic
nrow(ptitanic)
## [1] 1309

# calculamos el 75%
num_reg_entrena<-as.integer(0.75*nrow(ptitanic))
# Creamos una muestra aleatoria de registros de entrenamiento
v_titanic_train <- sample(nrow(ptitanic), num_reg_entrena)
# Creamos el conjunto de registros de entrenamiento
titanic_train <- ptitanic[v_titanic_train,]
head(titanic_train)

##      pclass survived    sex age sibsp parch
## 562      2nd      died   male  59     0     0
## 1077     3rd      died female NA     0     0
## 202      1st      died   male  54     0     0
## 329      2nd      died   male  34     1     0
## 1107     3rd      died female 41     0     5
## 1219     3rd      died   male  30     0     0

# Creamos los datos de comprobación o test (nótese el -)
titanic_test <- ptitanic[-v_titanic_train,]
```

Vamos a crear el modelo y entrenarlo

```
# Creamos un modelo de supervivencia en el Titanic

sobrevive.model <- rpart(survived ~ ., data = titanic_train, cp = .02)

# ahora hacemos predicciones sobre el grupo de test
titanic_test$pred <- predict(sobrevive.model,titanic_test, type = "class")
head(titanic_test)
##      pclass survived    sex    age sibsp parch    pred
## 2      1st survived   male  0.9167     1     2 survived
## 5      1st      died female 25.0000     1     2 survived
## 6      1st survived   male 48.0000     0     0      died
## 7      1st survived female 63.0000     1     0 survived
## 11     1st      died   male 47.0000     1     0      died
## 13     1st survived female 24.0000     0     0 survived
# Examinamos los resultados con la matriz de confusión
table(titanic_test$pred,titanic_test$survived)
##
##           died survived
## died       193       66
## survived   10       59
# Calculamos la bondad del modelo sobre el grupo de test
100*mean(titanic_test$pred==titanic_test$survived)
## [1] 76.82927
```

Como vemos el 77 % de ajuste es un valor alto, pero no infalible.

2.2.2 Poda de los árboles

Dada la facilidad con la que un árbol se complica, muchos paquetes tienen funciones especiales para cortar, limitar y optimizar el tamaño y la forma de los árboles. Por ejemplo, con la librería `rpart` lo podemos hacer limitando la profundidad del árbol y el número de divisiones máximo.

El proceso puede hacerse antes o después de crear el árbol, en lo que llamamos pre y post poda de control. En concreto la librería `rpart` contiene un parámetro que hemos estado usando el `cp`, que controla la complejidad del árbol.

```
# Ejemplo de pre-poda en rpart

require(rpart)

control.poda <- rpart.control(maxdepth = 2, minsplit = 10)

Titanic.model <- rpart(survived ~ .,
                      data = titanic_train,
                      method = "class",
                      control = control.poda)

rpart.plot(Titanic.model)

# cambiando los parámetros de poda el modelo es diferente:

control.poda <- rpart.control(maxdepth = 4)
Titanic.model <- rpart(survived ~ .,
                      data = titanic_train,
                      method = "class",
                      control = control.poda)

rpart.plot(Titanic.model)

# Ejemplo de post poda

# el parámetro cp, controla la post poda
# podemos ver su influencia dibujando la gráfica de cp

plotcp(Titanic.model)

# y simplificar el modelo anterior ya calculado
# como apreciamos a partir de cp=0.1 el modelo se simplifica mucho

rpart.plot(prune(Titanic.model, cp = 0.10))
```

2.3 Predicción con Análisis de Regresión.

2.3.1 Regresión Logística

Análisis de regresión utilizado para predecir el resultado de una variable categórica (aquella que puede adoptar un número limitado de categorías) en función de las variables predictoras (Villalba, 2018c).

Este modelo se enmarca dentro de los modelos denominados de *predicción lineal generalizados* o *glm* como son conocidos por sus siglas en inglés. Con el adjetivo binario nos referimos a las predicciones sobre variables binarias o dicotómicas que simplemente tratan de decir si algo es 1 o 0, SI o NO.

Este modelo de pronóstico se usa mucho en variables que se distribuyen en forma de binomial. La binomial es una distribución de probabilidad discreta que cuenta el número de éxitos en una secuencia de n ensayos.

Si el evento de éxito tiene una probabilidad de ocurrencia p , la probabilidad del evento contrario -el de fracaso- tendrá una probabilidad de $q=1-p$. En la distribución binomial se repite el experimento de éxito -fracaso n veces, de forma independiente, y se trata de calcular la probabilidad de un determinado número de éxitos d , en esas n repeticiones $B(n,p)$.

La denominación de logística se debe precisamente a la forma de la propia función de distribución de probabilidad binomial que presenta un crecimiento exponencial y que se parece a una S y que toma el nombre matemático de función logística $\frac{1}{1+e^{-t}}$.

Esta curva, es una aproximación continua a la función discreta binaria, pues el cambio de 0 a 1 se produce en corto espacio y muy pronunciado. Si usáramos otras funciones como la lineal para la regresión de datos binarios funcionaría muy mal, pues el ajuste lineal no capta bien la forma de los datos, las dos agrupaciones que buscamos separar o clasificar.

Los modelos de regresión logísticos se generan con la función `glm()` del paquete base R stats, de la siguiente manera.

```
m <- glm(y ~ x1 + x2 + x3,
        data = my_dataset,
        family = "binomial")

prob <- predict(m, test_dataset, type = "response")

pred <- ifelse(prob > 0.50, 1, 0)
```

Importante reseñar que la predicción se da en **modo de probabilidad**, por lo que, para evaluar un pronóstico concreto, se debe establecer qué umbral es el que fija el pronóstico 0 o 1. En el caso del ejemplo anterior se ha determinado que para $\text{pred} > 0,5$ el pronóstico es 1.

2.3.1.1 Construir modelos glm

Siguiendo con el uso de la base de datos de ejemplo de supervivientes del Titanic, vamos a crear un modelo logístico que pronostique la variable *Survived*. Pueden acceder a este enlace para ver como se crearon los datos en el apartado de [particiones de los datos](#).

Al igual que todos los modelos de aprendizaje, el modelo se compone de una fórmula, y luego se pronostica con la función `predict()`. En los modelos `glm()`, los únicos argumentos de `predict()` son `response` y `terms`. El primer caso da directamente la probabilidad de la respuesta y el segundo argumento proporciona los coeficientes de cada término en la fórmula. Si solo queremos obtener un valor de predicción usaremos `type = "response"`.

```
# Antes hemos cargado los datos del Titanic
# revisamos los datos
head(Titanic_data)
##      Class Sex  Age Survived
## 3      3rd Male Child       No
## 3.1    3rd Male Child       No
## 3.2    3rd Male Child       No
## 3.3    3rd Male Child       No
## 3.4    3rd Male Child       No
## 3.5    3rd Male Child       No
table(Titanic_data$Survived)
##
##      No  Yes
## 1490  711
```

```
# creamos una partición para crear un conjunto de test y otro de entrenamiento
library(caret)
## Loading required package: lattice
## Loading required package: ggplot2
set.seed(123)

# creamos un vector de partición sobre la variable Survived
# el tamaño de muestra será de 75%

trainIndex=createDataPartition(Titanic_data$Survived, p=0.70)$Resample1

# definamos los dos conjuntos de muestra

d_titanic_train=Titanic_data[trainIndex, ] # conjunto entrenamiento
d_titanic_test= Titanic_data[-trainIndex, ] # conjunto de test
```

Una vez tenemos los conjuntos de test y de aprendizaje creamos el modelo, usando la misma simbología que en el caso de los modelos de naive_bayes. La peculiaridad de glm() es que tenemos que identificar un umbral de probabilidad a partir del que consideramos el pronóstico 0 o 1.

```
# Construimos el modelo de predicción con la función glm
m_glm <- glm(Survived ~ Class+Sex, data = d_titanic_train, family =
"binomial")
# resumen del modelo
summary(m_glm)
##
## Call:
## glm(formula = Survived ~ Class + Sex, family = "binomial", data =
d_titanic_train)
##
## Deviance Residuals:
##      Min       1Q   Median       3Q      Max
## -2.1346  -0.7499  -0.4644   0.7435   2.1356
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  -0.2856     0.1678  -1.702   0.0888 .
## Class2nd      -1.0257     0.2352  -4.362 1.29e-05 ***
## Class3rd      -1.8870     0.2093  -9.017 < 2e-16 ***
## ClassCrew     -0.8394     0.1911  -4.393 1.12e-05 ***
## SexFemale      2.4557     0.1698  14.463 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 1939.3  on 1540  degrees of freedom
## Residual deviance: 1560.7  on 1536  degrees of freedom
## AIC: 1570.7
##
## Number of Fisher Scoring iterations: 4
```

```
# vemos las predicciones en el conjunto de test
d_titanic_test$pred<-predict(m_glm, d_titanic_test, type= "response")

# Hacemos el resumen gráfico del resultado
hist(100*d_titanic_test$pred, col="skyblue",
     main=" resultados modelo glm() sobre datos Titanic test",
     xlab="Probabilidad en % de supervivencia",
     ylab="Frecuencia")
# Marcamos un umbral en el que consideramos el pronóstico como donación
# este umbral lo ponemos en un valor del 60%
abline(v= 60,col= "navy", lwd=3) # marcamos el umbral de supervivencia
```

resultados modelo glm() sobre datos Titanic test

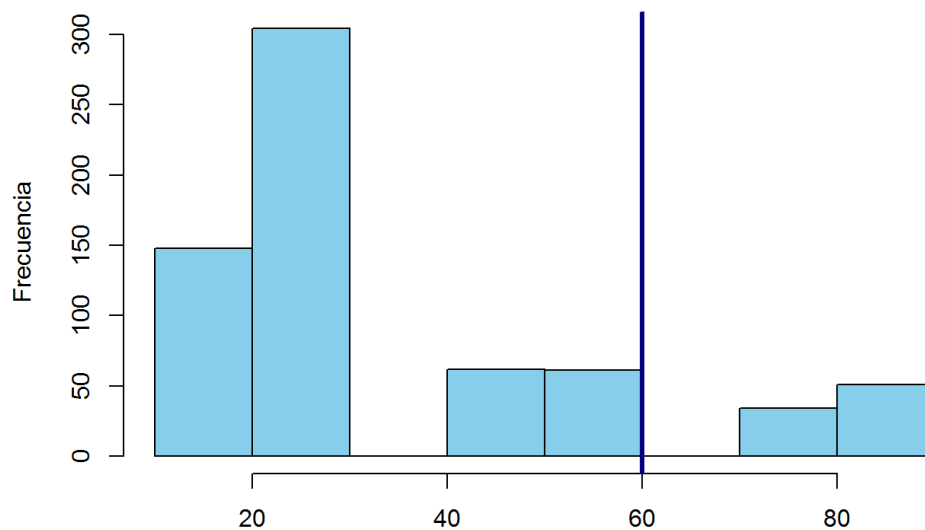


Figura 9. Probabilidad en porcentaje de supervivencia.

```
d_titanic_test$pred_final_60 <- ifelse(d_titanic_test$pred > 0.6, 1, 0)
# resumen de resultados
table(d_titanic_test$pred_final_60)
##
##    0    1
## 575   85
# podemos calcular el ajuste respecto a los casos reales con esta sencilla
# formula
# antes vamos a cambiar los levels de survived No=0, Yes=1
table(d_titanic_test$Survived) # vemos cual es el primero ---> No
##
## No Yes
## 447 213
levels(d_titanic_test$Survived) <- c(0,1)
mean(d_titanic_test$pred_final_60 == d_titanic_test$Survived)
## [1] 0.7878788
```

Como vemos una vez realizado el pronóstico podríamos probar diferentes umbrales y ver cuál es el que da un mejor resultado con esta metodología.

Curvas ROC y AUC

Estas curvas nos ayudan a controlar el acierto o no de los modelos cuando uno de los eventos es muy raro. Esto implica que predecir el evento opuesto conlleva un gran porcentaje de aciertos, y en cierta forma falsea la utilidad real de la predicción lo que hay que vigilar y entender. En estos casos es mejor sacrificar los aciertos generales en favor de concentrarlos sobre uno de los resultados, el más raro, el que buscamos distinguir. Por lo tanto, la exactitud de la predicción general es una medida engañosa en el rendimiento de lo que realmente nos interesa. Este es un caso muy común en predicciones binomiales pues un caso, el de éxito puede tener una probabilidad general mucho menor que el de fracaso, y un porcentaje de acierto elevado, puede no tener importancia, pues lo que nos interesa no es acertar los fracasos sino los éxitos.

Las curvas ROC son buenas para evaluar este problema en conjuntos de datos desequilibrados. Al hacer una gráfica **ROC** se representa mejor la compensación entre un modelo que es demasiado agresivo y uno que es demasiado pasivo. Lo que interesa es que el área de la curva sea máxima, cercana a 1, por lo que cuanto más se eleve respecto de la línea media mejor. Estas gráficas se pintan con la librería **pROC**. Usaremos dos funciones una para pintar la gráfica y otra que calcula el *AUC* o área bajo la curva.

```
# Cargamos La Librería de graficos ROC
library(pROC)

## Type 'citation("pROC")' for a citation.
##
## Attaching package: 'pROC'
## The following objects are masked from 'package:stats':
##
##     cov, smooth, var

# Creamos una curva ROC basada en el modelo glm anterior

ROC_glm60 <- roc(d_titanic_test$Survived, d_titanic_test$pred_final_60)

# Pintamos La gráfica ROC
plot(ROC_glm60, col = "blue")
```

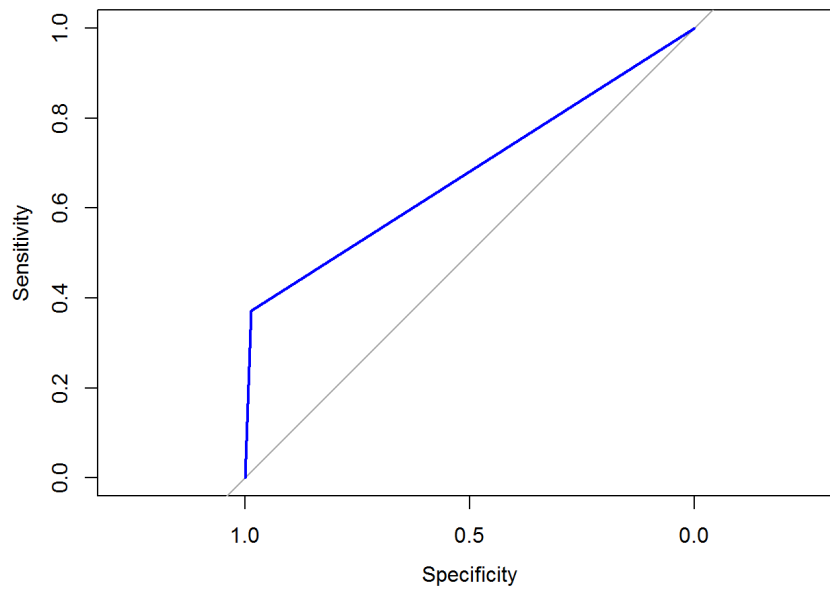



Figura 10. Gráfica ROC.

```
#plot(ROC_naive, col = "red")

# Calculamos el área bajo la ROC(AUC)
auc(ROC_glm60)
## Area under the curve: 0.6787
d_titanic_test$pred_final_40 <- ifelse(d_titanic_test$pred > 0.4, 1, 0)
ROC_glm40 <- roc(d_titanic_test$Survived, d_titanic_test$pred_final_40)
# Pintamos la gráfica ROC
plot(ROC_glm40, col = "red")
```

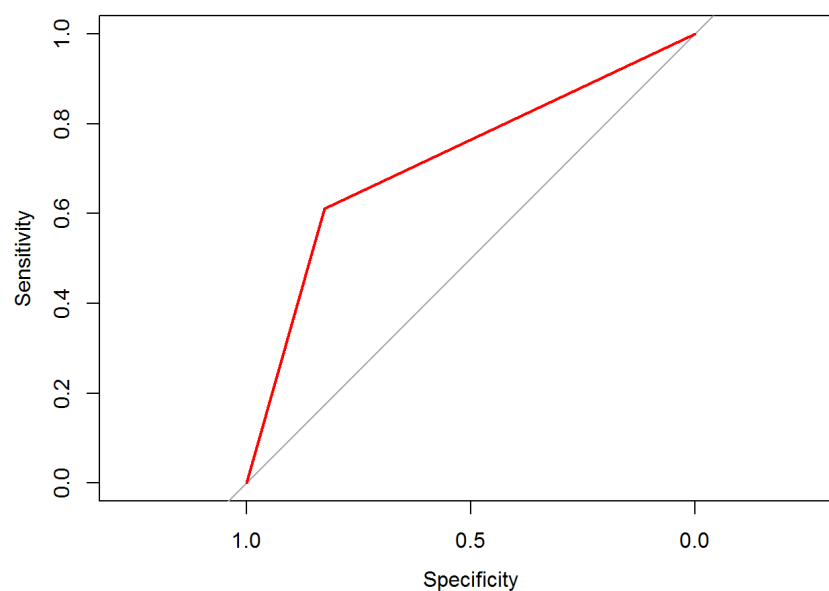


Figura 11. Gráfica ROC.

```
auc(ROC_glm40)
## Area under the curve: 0.7179
```

Vistos los resultados, el seleccionar un umbral de 40, mejora la predicción de casos positivos de supervivencia.

Optimización de un modelo glm

Cuando a priori no sabemos qué variables tienen más dependencia para crear el modelo una forma de hacerlo es usando la regresión gradual. Esto consiste en aplicar una función que va incrementando las variables y detecta el mejor modelo de regresión.

Para construirlo hacemos lo siguiente:

1. Creamos un modelo `glm()` sin predictores. Se hace estableciendo la variable explicativa igual a 1.
2. Se crea otro modelo con todas las variables usando `~.`
3. Se aplica la función `step()` entre ambos modelos para realizar una regresión progresiva hacia adelante. Debe indicarse la dirección con `direction = "forward"`
- 4 Usamos la función `predict()` sobre la lista de modelos creados con `step`

Veamos el ejemplo:

```
# 1. Modelo sin predictores
null_model <- glm(donated ~1, data = donors, family = "binomial")

# 2. modelo completo
full_model <- glm(donated ~ ., data = donors, family = "binomial")

# 3. funcion step ()
step_model <- step(null_model, scope = list(lower = null_model, upper =
full_model), direction = "forward")

## Start: AIC=37332.13
## donated ~ 1
## Warning in add1.glm(fit, scope$add, scale = scale, trace = trace, k = k, :
## using the 70916/93462 rows from a combined fit
##               Df Deviance   AIC
## + frequency      1    28502 37122
```

```

## + money          1      28621 37241
## + wealth_rating  1      28705 37326
## + has_children   1      28705 37326
## + age            1      28707 37328
## + interest_veterans 1      28709 37330
## + catalog_shopper 1      28710 37330
## + pet_owner      1      28711 37331
## <none>           1      28714 37332
## + interest_religion 1      28712 37333
## + recency        1      28713 37333
## + bad_address    1      28714 37334
## + veteran        1      28714 37334
##

## Step: AIC=37024.77
## donated ~ frequency
## Warning in add1.glm(fit, scope$add, scale = scale, trace = trace, k = k, :
## using the 70916/93462 rows from a combined fit
##              Df Deviance   AIC
## + money          1      28441 36966
## + wealth_rating  1      28493 37018
## + has_children   1      28494 37019
## + interest_veterans 1      28498 37023
## + catalog_shopper 1      28499 37024
## + age            1      28499 37024
## + pet_owner      1      28499 37024
## <none>           1      28502 37025
## + interest_religion 1      28501 37026
## + recency        1      28501 37026
## + bad_address    1      28502 37026
## + veteran        1      28502 37027
##

## Step: AIC=36949.71
## donated ~ frequency + money
## Warning in add1.glm(fit, scope$add, scale = scale, trace = trace, k = k, :
## using the 70916/93462 rows from a combined fit
##              Df Deviance   AIC
## + wealth_rating  1      28431 36942
## + has_children   1      28432 36943
## + interest_veterans 1      28438 36948
## + catalog_shopper 1      28438 36949
## + age            1      28439 36949
## + pet_owner      1      28439 36949
## <none>           1      28441 36950
## + interest_religion 1      28440 36951
## + recency        1      28441 36951
## + bad_address    1      28441 36951
## + veteran        1      28441 36952
##

## Step: AIC=36945.26
## donated ~ frequency + money + wealth_rating
## Warning in add1.glm(fit, scope$add, scale = scale, trace = trace, k = k, :
## using the 70916/93462 rows from a combined fit
##              Df Deviance   AIC
## + has_children   1      28421 36937
## + interest_veterans 1      28429 36945
## + catalog_shopper 1      28429 36945
## + age            1      28429 36945
## <none>           1      28431 36945

```

```

## + pet_owner          1      28430 36945
## + interest_religion  1      28431 36947
## + recency            1      28431 36947
## + bad_address        1      28431 36947
## + veteran            1      28431 36947
##
## Step: AIC=36938.08
## donated ~ frequency + money + wealth_rating + has_children
## Warning in add1.glm(fit, scope$add, scale = scale, trace = trace, k = k, :
## using the 70916/93462 rows from a combined fit
##              Df Deviance   AIC
## + pet_owner      1      28418 36937
## + catalog_shopper 1      28418 36937
## + interest_veterans 1      28418 36937
## <none>              28421 36938
## + interest_religion 1      28420 36939
## + recency          1      28421 36940
## + age               1      28421 36940
## + bad_address       1      28421 36940
## + veteran           1      28421 36940
##
## Step: AIC=36932.08
## donated ~ frequency + money + wealth_rating + has_children +
## pet_owner
## Warning in add1.glm(fit, scope$add, scale = scale, trace = trace, k = k, :
## using the 70916/93462 rows from a combined fit
##              Df Deviance   AIC
## <none>              28418 36932
## + interest_veterans 1      28416 36932
## + catalog_shopper   1      28416 36932
## + age                1      28417 36933
## + recency            1      28417 36934
## + interest_religion 1      28417 36934
## + bad_address        1      28418 36934
## + veteran            1      28418 36934
## summary(step_model)
##
## Call:
## glm(formula = donated ~ frequency + money + wealth_rating + has_children +
## pet_owner, family = "binomial", data = donors)
##
## Deviance Residuals:
##      Min       1Q   Median       3Q      Max
## -0.4023  -0.3625  -0.2988  -0.2847   2.7328
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)   -3.05529    0.04556 -67.058 < 2e-16 ***
## frequencyINFREQUENT -0.49649    0.03100 -16.017 < 2e-16 ***
## moneyMEDIUM      0.36594    0.04301   8.508 < 2e-16 ***
## wealth_rating    0.03294    0.01238   2.660 0.007805 **
## has_children    -0.15820    0.04707  -3.361 0.000777 ***
## pet_owner       0.11712    0.04096   2.860 0.004243 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 37330  on 93461  degrees of freedom

```

```
## Residual deviance: 36920  on 93456  degrees of freedom
## AIC: 36932
##
## Number of Fisher Scoring iterations: 6
# estimamos la probabilidad
step_prob <- predict(step_model, type = "response")

# Pintamos ROC of the stepwise model
library(pROC)
ROC <- roc(donors$donated, step_prob)
plot(ROC, col = "red")
```

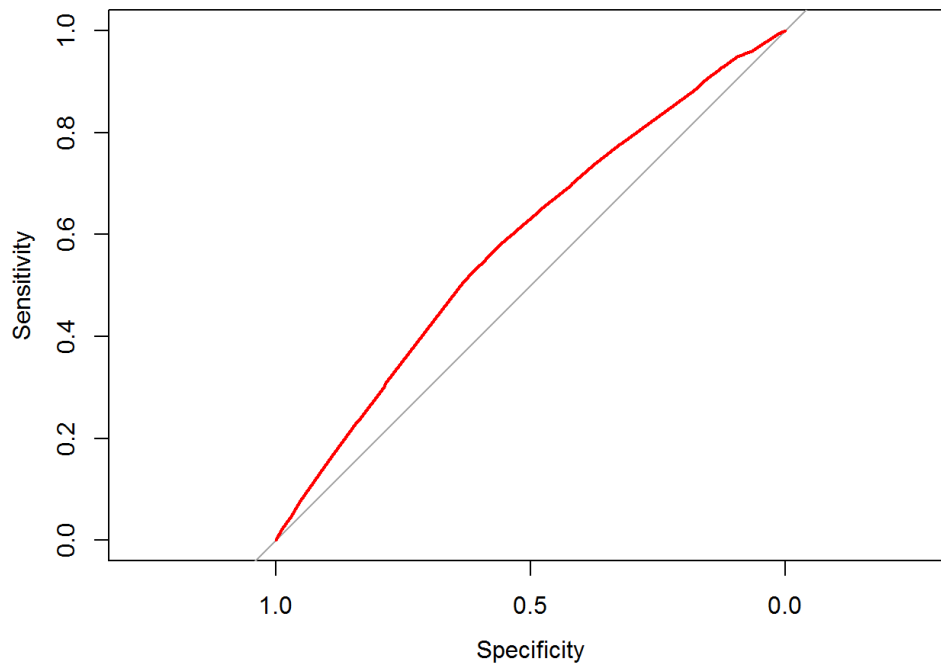


Figura 12. Gráfica ROC.

```
auc(ROC)
## Area under the curve: 0.585
```

OTROS EJEMPLOS

Ejemplo 1. Datos de crédito en Alemania (**Credit scoring**) (Dae-Jin, 2019).

Cuando un banco recibe una solicitud de préstamo, basada en el perfil del solicitante, el banco tiene que tomar una decisión sobre si procede o no con la aprobación del préstamo. Hay dos tipos de riesgos asociados a la decisión del banco:

- Si el solicitante tiene un buen riesgo de crédito, es decir, es probable que pague el préstamo, entonces no aprobar el préstamo a la persona resulta en una pérdida de negocio para el banco.
- Si el solicitante tiene un riesgo de crédito malo, es decir, no es probable que pague el préstamo, entonces la aprobación del préstamo a la persona resulta en una pérdida financiera para el banco.

El objetivo de este tipo de análisis es **minimizar el riesgo y maximizar el beneficio del banco**. Para minimizar las pérdidas desde la perspectiva del banco, el banco necesita una regla de decisión sobre a quién dar la aprobación del préstamo y a quién no. Los perfiles demográficos y socioeconómicos de un solicitante son considerados por los administradores de préstamos antes de que se tome una decisión sobre su solicitud de préstamo.

Los datos German Credit Data contiene datos sobre 20 variables y la clasificación de si un solicitante es considerado un *Bueno* o un *Malo* riesgo de crédito para 1000 solicitantes de préstamos. Datos disponibles en:

[https://archive.ics.uci.edu/ml/datasets/statlog+\(german+credit+data\)](https://archive.ics.uci.edu/ml/datasets/statlog+(german+credit+data))

Se espera que un *modelo predictivo* elaborado a partir de estos datos sirva de guía al gerente del banco para tomar una decisión sobre la aprobación de un préstamo a un posible solicitante en función de su perfil.

```
# German Credit Data
gcreditdata <- read.table("https://archive.ics.uci.edu/ml/machine-learning-
databases/statlog/german/german.data", header = FALSE)

colnames(gcreditdata) <- c("account.status", "months",
                           "credit.history", "purpose", "credit.amount",
                           "savings", "employment", "installment.rate", "personal.status",
                           "guarantors", "residence", "property", "age", "other.installments",
                           "housing", "credit.cards", "job", "dependents", "phone", "foreign.worker", "credit.ra
ting")

head(gcreditdata)

##   account.status months credit.history purpose credit.amount savings
## 1           A11      6           A34    A43         1169      A65
## 2           A12     48           A32    A43         5951      A61
## 3           A14     12           A34    A46         2096      A61
```

```
## 4      A11      42      A32      A42      7882      A61
## 5      A11      24      A33      A40      4870      A61
## 6      A14      36      A32      A46      9055      A65
## employment installment.rate personal.status guarantors residence
## 1      A75              4              A93      A101      4
## 2      A73              2              A92      A101      2
## 3      A74              2              A93      A101      3
## 4      A74              2              A93      A103      4
## 5      A73              3              A93      A101      4
## 6      A73              2              A93      A101      4
## property age other.installments housing credit.cards job dependents
## 1      A121  67              A143  A152      2 A173      1
## 2      A121  22              A143  A152      1 A173      1
## 3      A121  49              A143  A152      1 A172      2
## 4      A122  45              A143  A153      1 A173      2
## 5      A124  53              A143  A153      2 A173      2
## 6      A124  35              A143  A153      1 A172      2
## phone foreign.worker credit.rating
## 1 A192              A201      1
## 2 A191              A201      2
## 3 A191              A201      1
## 4 A191              A201      1
## 5 A191              A201      2
## 6 A192              A201      1
```

Cambie el nombre de algunos niveles de factor:

```
table(gcreditdata$credit.rating) # Good = 1 / Bad = 2
##
## 1 2
## 700 300
gcreditdata$credit.rating <- ifelse(gcreditdata$credit.rating==1,1,0)
table(gcreditdata$credit.rating) # Good = 1 / Bad = 0
##
## 0 1
## 300 700
levels(gcreditdata$account.status) <- c("<0DM", "<200DM", ">200DM", "NoStatus")
levels(gcreditdata$credit.history) <-
c("No", "Allpaid", "Allpaidtillnow", "Delayinpaying", "Critical")
levels(gcreditdata$purpose) <-
c("car(new)", "car(used)", "furniture/equipment", "radio/television", "domesticappl
iances",
"repairs", "education", "vacation-
doesnotexist?", "retraining", "business", "others")
```

Dividamos los datos en dos grupos (entrenamiento y prueba), 70%/30%.

```
n<- dim(gcreditdata)[1]

set.seed(1234) # select a random sample with
train <- sample(1:n , 0.7*n)
```

```
gcreditdata.test <- gcreditdata[-train,]
gcreditdata.train <- gcreditdata[train,]

ytrain <- gcreditdata$credit.rating[train]
ytest <- gcreditdata$credit.rating[-train]
```

Un modelo logístico puede ser ajustado con la función glm.

```
m1 <- glm(credit.rating ~ . , family = binomial, data= gcreditdata.train)

summary(m1)
```

```
##
## Call:
## glm(formula = credit.rating ~ . , family = binomial, data =
## gcreditdata.train)
##
## Deviance Residuals:
##      Min       1Q   Median       3Q      Max
## -2.8353  -0.6178   0.3415   0.6729   2.0514
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)      5.259e-01  1.371e+00   0.384 0.701257
## account.status<200DM  3.532e-01  2.719e-01   1.299 0.193876
## account.status>200DM  8.198e-01  4.626e-01   1.772 0.076353 .
## account.statusNoStatus 1.487e+00  2.782e-01   5.344 9.08e-08 ***
## months           -3.687e-02  1.156e-02  -3.189 0.001428 **
## credit.historyAllpaid -1.347e-01  6.539e-01  -0.206 0.836779
## credit.historyAllpaidtillnow 5.296e-01  4.975e-01   1.065 0.287038
## credit.historyDelayinpaying 7.156e-01  5.575e-01   1.284 0.199266
## credit.historyCritical 1.439e+00  5.073e-01   2.837 0.004559 **
## purposecar(used)      1.838e+00  4.856e-01   3.786 0.000153 ***
## purposefurniture/equipment 1.095e+00  9.231e-01   1.186 0.235728
## purposeradio/television 8.879e-01  3.289e-01   2.700 0.006944 **
## purposedomesticappliances 9.928e-01  3.107e-01   3.196 0.001395 **
## purposerepairs        5.399e-01  9.006e-01   0.599 0.548881
## purposeeducation     -4.889e-01  6.587e-01  -0.742 0.457933
## purposevacation-doesnotexist? -1.800e-01  4.648e-01  -0.387 0.698616
## purposeretraining     2.093e+00  1.266e+00   1.653 0.098417 .
## purposebusiness      5.772e-01  3.985e-01   1.448 0.147489
## credit.amount        -1.107e-04  5.456e-05  -2.028 0.042526 *
## savingsA62           5.219e-01  3.548e-01   1.471 0.141326
## savingsA63           8.722e-01  5.710e-01   1.528 0.126623
## savingsA64           1.746e+00  6.533e-01   2.673 0.007514 **
## savingsA65           1.354e+00  3.350e-01   4.042 5.30e-05 ***
## employmentA72        1.729e-01  5.327e-01   0.324 0.745566
## employmentA73        2.462e-01  5.116e-01   0.481 0.630356
## employmentA74        1.182e+00  5.660e-01   2.089 0.036736 *
## employmentA75        4.924e-02  5.076e-01   0.097 0.922712
## installment.rate     -4.157e-01  1.098e-01  -3.787 0.000152 ***
## personal.statusA92    5.279e-02  4.718e-01   0.112 0.910902
## personal.statusA93    6.791e-01  4.577e-01   1.484 0.137906
## personal.statusA94    2.017e-01  5.550e-01   0.363 0.716253
## guarantorsA102       1.880e-01  5.068e-01   0.371 0.710613
```



```
## guarantorsA103      1.024e+00  5.814e-01   1.761 0.078259 .
## residence           5.357e-02  1.047e-01   0.512 0.608926
## propertyA122       -4.618e-01  3.142e-01  -1.470 0.141584
## propertyA123       -4.679e-01  2.906e-01  -1.610 0.107366
## propertyA124       -7.642e-01  5.658e-01  -1.351 0.176834
## age                1.872e-02  1.172e-02   1.598 0.110121
## other.installmentsA142 -1.259e-01  5.197e-01  -0.242 0.808648
## other.installmentsA143  3.400e-01  2.892e-01   1.176 0.239669
## housingA152         5.768e-01  2.935e-01   1.965 0.049384 *
## housingA153         7.825e-01  6.261e-01   1.250 0.211387
## credit.cards        -3.720e-01  2.456e-01  -1.515 0.129882
## jobA172             -8.402e-01  8.307e-01  -1.011 0.311822
## jobA173             -9.497e-01  7.970e-01  -1.192 0.233388
## jobA174            -9.513e-01  8.262e-01  -1.152 0.249525
## dependents         -2.108e-01  3.080e-01  -0.685 0.493620
## phoneA192           4.876e-01  2.556e-01   1.908 0.056400 .
## foreign.workerA202    9.540e-01  7.550e-01   1.264 0.206386
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 839.40  on 699  degrees of freedom
## Residual deviance: 596.11  on 651  degrees of freedom
## AIC: 694.11
##
## Number of Fisher Scoring iterations: 5
```

Variables significativas:

```
sig.var<- summary(m1)$coeff[-1,4] <0.01

names(sig.var)[sig.var == TRUE]
## [1] "account.statusNoStatus"      "months"
## [3] "credit.historyCritical"       "purposecar(used)"
## [5] "purposeradio/television"     "purposedomesticappliances"
## [7] "savingsA64"                  "savingsA65"
## [9] "installment.rate"
```

Con predict podemos predecir con el modelo logístico el conjunto de test.

```
pred1<- predict.glm(m1,newdata = gcreditdata.test, type="response")
result1<- table(ytest, floor(pred1+0.5))
result1

##
## ytest    0    1
##      0  48  51
##      1  25 176

error1<- sum(result1[1,2], result1[2,1])/sum(result1)
error1

## [1] 0.2533333
```

Curva ROC con la librería ROCR:

```
library(ROCR)
## Loading required package: gplots
##
## Attaching package: 'gplots'
## The following object is masked from 'package:stats':
##
##     lowess
pred = ROCR::prediction(pred1,ytest)
perf <- performance(pred, "tpr", "fpr")
plot(perf)
```

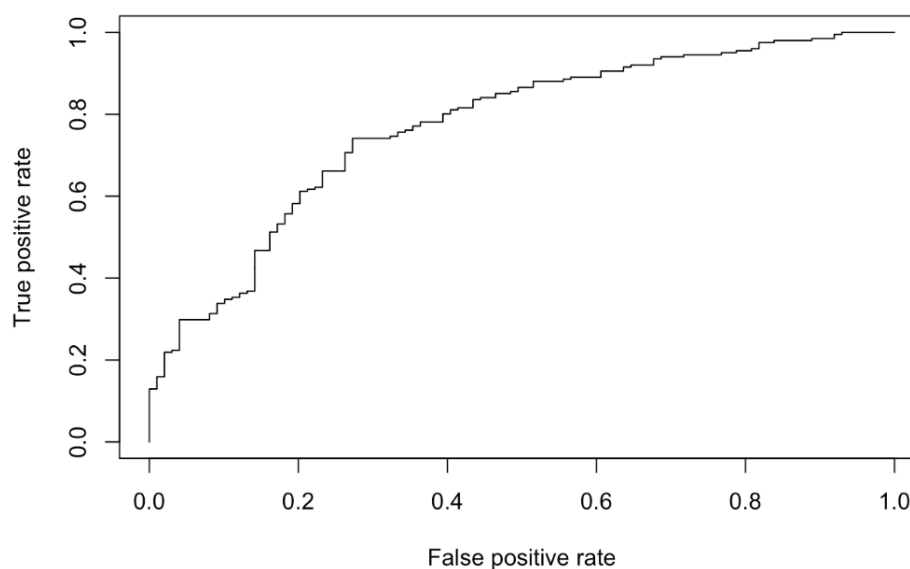


Figura 13. Gráfica ROC.

```
AUCLog1=performance(pred, measure = "auc")@y.values[[1]]
cat("AUC: ",AUCLog1,"n")
## AUC: 0.7699382 n
```

Ejemplo 2: Predecir el salario de los trabajadores (Dae-Jin, 2019).

Consideremos los datos `adult.csv`. Intentaremos predecir la variable de respuesta `ABOVE50k` (Salario >50k) mediante una regresión logística basada en variables demográficas explicativas.

```
inputData <- read.csv("http://idaejin.github.io/courses/R/data/adult.csv")
head(inputData)
##   AGE      WORKCLASS FNLWGT  EDUCATION EDUCATIONNUM      MARITALSTATUS
## 1  39    State-gov   77516  Bachelors         13    Never-married
## 2  50 Self-emp-not-inc 83311  Bachelors         13  Married-civ-spouse
## 3  38      Private  215646   HS-grad          9      Divorced
## 4  53      Private  234721    11th           7  Married-civ-spouse
## 5  28      Private  338409  Bachelors         13  Married-civ-spouse
## 6  37      Private  284582   Masters         14  Married-civ-spouse
##              OCCUPATION  RELATIONSHIP  RACE    SEX CAPITALGAIN CAPITALLOSS
## 1      Adm-clerical  Not-in-family  White  Male      2174          0
## 2      Exec-managerial      Husband  White  Male          0          0
## 3  Handlers-cleaners  Not-in-family  White  Male          0          0
## 4  Handlers-cleaners      Husband  Black  Male          0          0
## 5      Prof-specialty      Wife  Black  Female          0          0
## 6      Exec-managerial      Wife  White  Female          0          0
##   HOURSPERWEEK  NATIVECOUNTRY ABOVE50K
## 1           40  United-States          0
## 2           13  United-States          0
## 3           40  United-States          0
## 4           40  United-States          0
## 5           40          Cuba          0
## 6           40  United-States          0
```

Verificar sesgo de clase

Idealmente, la proporción de eventos y no eventos en la variable YY debería ser aproximadamente la misma. Por lo tanto, primero verifiquemos la proporción de clases en la variable dependiente ABOVE50K.

```
table(inputData$ABOVE50K)
```

```
##
```

```
##   0   1
```

```
## 24720 7841
```

Claramente, existe un sesgo de clase, una condición observada cuando la proporción de eventos es mucho menor que la proporción de no eventos. Por lo tanto, debemos muestrear las observaciones en proporciones aproximadamente iguales para obtener mejores modelos.

Crear Muestras de Entrenamiento y Pruebas

Una manera de abordar el problema del sesgo de clase es muestrear los 0 y 1 para los trainingData (muestra de entrenamiento) en proporciones iguales. Al hacerlo, pondremos el resto de los inputData no incluidos para la formación en testData (muestra de validación). Como resultado, el tamaño de la muestra de entrenamiento será menor que el de la validación, lo que está bien, porque hay un gran número de observaciones (>10K).

```
# Create Training Data
input_ones <- inputData[which(inputData$ABOVE50K == 1), ] # all 1's
input_zeros <- inputData[which(inputData$ABOVE50K == 0), ] # all 0's

set.seed(100) # for repeatability of samples

input_ones_training_rows <- sample(1:nrow(input_ones), 0.7*nrow(input_ones)) #
1's for training
input_zeros_training_rows <- sample(1:nrow(input_zeros), 0.7*nrow(input_ones))
# 0's for training.

# Pick as many 0's as 1's
training_ones <- input_ones[input_ones_training_rows, ]
training_zeros <- input_zeros[input_zeros_training_rows, ]
trainingData <- rbind(training_ones, training_zeros) # row bind the 1's and
0's

# Create Test Data
test_ones <- input_ones[-input_ones_training_rows, ]
test_zeros <- input_zeros[-input_zeros_training_rows, ]

testData <- rbind(test_ones, test_zeros) # row bind the 1's and 0's
```

Construir modelos de Logit y predecir

```
logitMod <- glm(ABOVE50K ~ RELATIONSHIP + AGE + CAPITALGAIN + OCCUPATION +
EDUCATIONNUM, data=trainingData, family=binomial(link="logit"))
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
predicted <- plogis(predict(logitMod, testData)) # predicted scores
# or
predicted <- predict(logitMod, testData, type="response") # predicted scores
```

Cuando usamos la función de predicción en este modelo, predecirá las probabilidades de la variable YY. Para convertirlo en una probabilidad de predicción que esté entre 0 y 1, usamos el plogis().

Decidir la probabilidad de corte de predicción óptima para el modelo.

El puntaje de probabilidad de la predicción de corte por defecto es de 0.50.5 o la proporción de 1's y 0's en los datos de entrenamiento. Pero a veces, afinar el corte de probabilidad puede mejorar la precisión tanto en las muestras de desarrollo como en las de validación. La función `InformationValue::optimalCutoff` proporciona formas de encontrar el punto de corte óptimo para mejorar la predicción de 1, 0, 1 y 0 y reducir el error de clasificación. Permite calcular la puntuación óptima que minimiza el error de clasificación para el modelo anterior.

```
library(InformationValue)
optCutoff <- optimalCutoff(testData$ABOVE50K, predicted)[1]
optCutoff
## [1] 0.89
```

Error de clasificación

El error de clasificación errónea es el desajuste porcentual de los valores predefinidos frente a los reales, independientemente de que sean 1 o 0. Cuanto menor sea el error de clasificación, mejor será su modelo.

```
misClassError(testData$ABOVE50K, predicted, threshold = optCutoff)
## [1] 0.0892
```

ROC Receiver Operating Characteristics

La curva traza el porcentaje de verdaderos positivos pronosticados con precisión por un modelo logit dado a medida que la probabilidad de corte de la predicción se reduce de 1 a 0. Para un buen modelo, a medida que se reduce el corte, debería marcar más de 1 real como positivo y menos de 0 real como 1. Por lo tanto, para un buen modelo, la curva debería subir bruscamente, indicando que el TPR (eje Y) aumenta más rápido que el FPR (eje X) a medida que disminuye la puntuación de corte. Cuanto mayor sea el área bajo la curva ROC, mejor será la capacidad de predicción del modelo.

```
plotROC(testData$ABOVE50K, predicted)
```

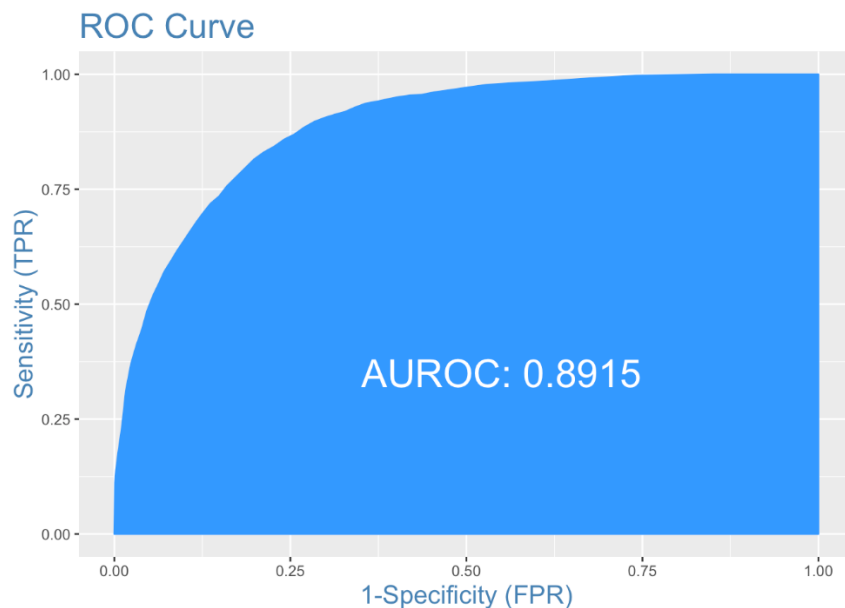


Figura 14. Gráfica ROC.

Especificidad y Sensibilidad

La **Sensibilidad** (o Tasa Verdaderos Positivos) es el porcentaje de 1's (reales) correctamente predichos por el modelo, mientras que, **especificidad** es el porcentaje de 0's (reales) correctamente predicho. La **especificidad** también puede calcularse como 1-Tasa Falsos Positivos.

Sensitivity = $\frac{\text{\#Actual 1's and Predicted as 1's}}{\text{\#Actual 1's}}$

Specificity = $\frac{\text{\#Actual 0's and Predicted as 0's}}{\text{\#Actual 0's}}$

```
sensitivity(testData$ABOVE50K, predicted, threshold = optCutOff)
## [1] 0.3442414
specificity(testData$ABOVE50K, predicted, threshold = optCutOff)
## [1] 0.9800853
```

Los números anteriores se calculan a partir de la muestra de validación que no se utilizó para la formación del modelo. Así que, una tasa de detección de la verdad de 34.42% en los datos de prueba es bueno.

Matriz de Confusión Las columnas son reales, mientras que las filas son predicciones.

```
confusionMatrix(testData$ABOVE50K, predicted, threshold = optCutOff)
##          0          1
## 0 18849 1543
## 1   383  810
```

Tema 3: Métricas de calidad en modelos predictivos.

3.1 Matriz de confusión.

Una matriz de confusión, también conocida como matriz de error, nos permite la visualización del rendimiento de un algoritmo, generalmente uno de aprendizaje supervisado (en el aprendizaje no supervisado generalmente se denomina matriz coincidente). Cada fila de la matriz representa las instancias en una clase predicha, mientras que cada columna representa las instancias en una clase real (o viceversa). El nombre se deriva del hecho de que hace que sea fácil ver si el sistema confunde dos clases (es decir, comúnmente etiquetar incorrectamente una como otra). Las clasificaciones binarias, cuando el *outcome* puede tomar solo dos clases, nos arrojan esta siguiente matriz de confusión.

	Realmente es positivo	Realmente es negativo
Predicho como positivo	Verdaderos Positivos (VP)	Falsos Positivos (FP)
Predicho como negativo	Falsos Negativos (FN)	Verdaderos Negativos (VN)

Figura 15. Matriz de confusión.

3.1.1 Exactitud.

La exactitud del modelo (accuracy en inglés), la podemos calcular a partir de la matriz de confusión:

$$Accuracy = \frac{VP+VN}{VP+VN+FP+FN}$$

El *accuracy* del modelo es la proporción de veces que el algoritmo predijo correctamente, respecto al total de datos evaluados.

3.1.2 Sensitividad.

La sensibilidad (también llamada tasa positiva verdadera, el recuerdo o la probabilidad de detección en algunos campos) mide la proporción de positivos reales que se identifican correctamente como tales (por ejemplo, el porcentaje de personas enfermas que se identifican correctamente como que tienen condición).

$$Sensitivity = \frac{VP}{VP+FN}$$

3.1.3 Especificidad.

La especificidad (también llamada tasa negativa verdadera) mide la proporción de negativos reales que se identifican correctamente como tales (por ejemplo, el porcentaje de personas sanas que se identifican correctamente como que no tienen la afección).

$$Specificity = \frac{VN}{VN+FP}$$

Ejercicio propuesto / Solución:

1. Utilizando la librería *caret*, particiona el data frame *iris* de tal forma de tener 70% de datos de entrenamiento y 30% de datos de prueba.

```
index_entrena <- createDataPartition(y = iris$Species, p = 0.7, list = FALSE)

iris2_entrena <- iris[index_entrena, ]
iris2_test <- iris[-index_entrena, ]
```


Bibliografía:

Agencia B12. (2020). *Qué es un modelo predictivo y cómo se aplica al negocio*.
<https://agenciab12.com/noticia/que-es-modelo-predictivo-como-aplica-negocio>

Arimetrics. (2022). *Modelo predictivo - Definición, significado y ejemplos*.
<https://www.arimetrics.com/glosario-digital/modelo-predictivo>

Calderón Méndez, N. de J. (2006). *Minería de datos, una herramienta para la toma de decisiones*.

Dae-Jin, L. (2019). *Regresión logística. Introducción al software estadístico R*.
<https://idaejin.github.io/courses/R/2019/euskaltel/regresion-logistica.html>

Espino Timón, C. (2017). *Análisis predictivo: técnicas y modelos utilizados y aplicaciones del mismo - herramientas Open Source que permiten su uso* [Universitat Oberta de Catalunya].
<http://openaccess.uoc.edu/webapps/o2/bitstream/10609/59565/6/caresptimTFG0117memòria.pdf>

Keyrus. (2022). *Las 11 técnicas más utilizadas en el modelado de análisis predictivos*.
<https://keyrus.com/sp/es/insights/las-11-tecnicas-mas-utilizadas-en-el-modelado-de-analisis-predictivos>

Paredes Inilupu, D. (2020). *Data Science con R. Análisis de Datos y algoritmos de predicción*.
<https://bookdown.org/dparedesi/data-science-con-r/aprendizaje-supervisado.html>

Parra, F. J. (2019). *Métodos de clasificación. Estadística y Machine Learning con R*.
 En *Estadística y Machine Learning con R*.
<https://bookdown.org/content/2274/metodos-de-clasificacion.html>

Rodríguez, D. (2018, noviembre). *Diferencias entre regresión y clasificación en aprendizaje automático*. <https://www.analyticslane.com/2018/11/26/diferencias-entre-regresion-y-clasificacion-en-aprendizaje-automatico/>

Terreros, D. (2021). *Análisis predictivo: etapas, modelos y herramientas para las organizaciones*. <https://blog.hubspot.es/marketing/que-es-analisis-predictivo>

Villalba, F. (2018a). *Árboles de decisión. Aprendizaje supervisado en R*.
<https://fervilber.github.io/Aprendizaje-supervisado-en-R/arboles.html>

Villalba, F. (2018b). Naive Bayes - Clasificación bayesiano ingenuo. *Aprendizaje supervisado en R*. <https://fervilber.github.io/Aprendizaje-supervisado-en-R/ingenuo.html>

Villalba, F. (2018c). *Regresión logística binaria. Aprendizaje supervisado en R*.
<https://fervilber.github.io/Aprendizaje-supervisado-en-R/glm.html>

Zúñiga Quiñones, L. M. (2021). *Desarrollo de un modelo predictivo para un sistema de manufactura orientado a la industria 4.0* [Pontificia Universidad Javeriana de Colombia].

[https://repository.javeriana.edu.co/bitstream/handle/10554/55631/123-attachment-1627079269Lina Zuñiga - Trabajo de Grado.pdf?sequence=4](https://repository.javeriana.edu.co/bitstream/handle/10554/55631/123-attachment-1627079269Lina%20Zu%C3%B1iga%20-%20Trabajo%20de%20Grado.pdf?sequence=4)